



**Arab Academy for Science, Technology and Maritime Transport
(AASTMT)
College of Artificial Intelligence**

B. SC. Final Year Project

**Artificial Intelligence mobile application for psychotherapy
(AICO)**

By:

Laith Khaled 20104060 (IS)
Marco Magdy 20102963 (IS)
Moris Sameh 20103787 (IS)

Nada Ahmed 20109955 (IS)
Salma Basem 20109951 (IS)

Supervised:

Prof. Dr. Ossama Muhamed Badawy

Dr. Muhamed Saad Zaghloul

July- 2024

DECLARATION

I hereby certify that this material, which I now submit for assessment on the program of study leading to the award of Bachelor of Science in Artificial Intelligence is entirely my own work, that I have exercised reasonable care to ensure that the work is original, and does not, to the best of my knowledge, breach any law of copyright, and has not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of my work.

Student Name	Registration Number	Signature	Date	Department
<i>Laith Khaled</i>	20104060	Laith	15/6	Intelligent systems and Robotics
<i>Marco Magdy</i>	20102963	Marco	14/6	Intelligent systems and Robotics
<i>Moris Sameh</i>	20103787	Moris	15/6	Intelligent systems and Robotics
<i>Nada Ahmad</i>	20109955	Nada	14/6	Intelligent systems and Robotics
<i>Salma Basem</i>	20109951	Salma	13/6	Intelligent systems and Robotics



Arab Academy for Science, Technology and Maritime Transport

College of Artificial Intelligence

Intelligent systems and Robotics department

Academic Year: 2024 Semester: 8

Graduation Project Summary Report

Project Title	Artificial Intelligence mobile application for psychotherapy	
Supervisor(s)	Prof. Dr. Ossama Muhamed Badawy & Dr. Muhamed Saad Zaghloul	
Team members	Registration Number	Name
	20104060	Laith Khaled Mohi Eldeen
	20102963	Marco Magdy Fayk
	20103787	Moris Sameh Moris
	20109955	Nada Ahmed Attia
	20109951	Salma Basem Elfeel
Project Deliverables	• A mobile application featuring an AI chatbot psychotherapist. • A fine-tuned large language model trained on psychotherapy data.	
Abstract	This project develops an AI-powered mobile application for psychotherapy, aiming to provide accessible, cost-effective mental	

	health support. Initial testing shows that the app provides responses like those of human therapists, highlighting its potential to transform psychotherapy services.	
System Constraints	English version only	
Project Impact	Improvement in mental health state	
Team Organization	Name	Tasks achieved
	Moris Sameh Moris Marco Magdy Fayek	Database Integration
	Marco Magdy Fayk Moris Sameh Moris	Large Language Model implementation, integration, Fine tuning and testing
	Salma Basem Elfeel Nada Ahmed Attia Moris Sameh Moris Laith Khaled Mohi Elden	Server Management
	Salma Basem Elfeel Nada Ahmed Attia Laith Khaled Mohi Elden	Front-End Mobile Application Development and testing
Ethics/Safety	Ensuring user privacy, security, validation, authentication and ethical AI use.	

Main Supervisors Signature

.....

ACKNOWLEDGMENT

I begin with heartfelt gratitude to ALLAH, whose blessings and guidance have made this thesis possible. I am profoundly thankful for the strength, patience, and willpower granted to me by ALLAH to complete this work.

My deepest appreciation goes to my esteemed supervisors, Prof. Dr. Osama Mohamed Badawy and Dr. Mohamed Saad Zaghloul. Their invaluable guidance, mentorship, and wisdom have significantly shaped this thesis and nurtured my academic growth. Their expertise, dedication, and encouragement have been pivotal in my journey.

I would also like to express my profound gratitude to my beloved family for their unwavering support, encouragement, and understanding throughout my academic journey. Their endless love, patience, and belief in me have been a constant source of strength and motivation.

Lastly, I extend my sincere appreciation to all those who have contributed to this thesis in any capacity, directly or indirectly. Your support and encouragement have been indispensable, and I am deeply grateful for your assistance.

ABSTRACT

This project explores the feasibility of employing a large language model (LLM) to develop a mobile application incorporating an AI-powered chatbot psychotherapist. The LLM was meticulously fine-tuned on a specialized psychotherapy dataset to facilitate therapeutic dialogues with users. Various challenges were addressed throughout the project, including the selection of an appropriate LLM and the creation of a comprehensive question-and-answer psychotherapy dataset. An innovative methodology leveraging LLM-generated data was utilized to construct a robust training dataset for fine-tuning the model.

Despite successfully developing a functional prototype, constraints in time and resources necessitated the deferment of deploying the model on a high-performance server and integrating it into a native Android application. The findings of this project underscore the potential of LLMs in the development of AI-driven therapeutic tools, highlighting the need for further exploration and refinement in future iterations.

TABLE OF CONTENTS

DECLARATION.....	ii
ABSTRACT	vi
TABLE OF CONTENTS.....	vii
LIST OF FIGURES	x
LIST OF TABLES:	xi
Chapter 1: INTRODUCTION	2
1.1 Overview:	2
1.2 Motivation:	3
1.3 Problem Statement:	3
1.4 Objectives:.....	4
1.5 Problem Complexity:	4
1.6 Constraints:	6
Chapter 2: RELATED WORK.....	10
2.1 Research Papers:.....	10
2.2 Competitors:	13
2.2.1 “WYSA”:.....	13
2.2.2 “Woebot”:.....	14
2.2.3 “Replika”:.....	15
2.2.4 “Mindspa”:	16
2.2.5 “Microsoft Bing”:.....	17
Chapter 3: SYSTEM ANALYSIS AND DESIGN:.....	19
3.1 Resource Requirements:.....	19
3.2 Product Features:	19
3.3 Operating Environment:	20
3.4 Functional Features:	20
3.5 Non-functional Features:	20
3.6 Diagrams:	22
3.6.1 Architecture Diagram	22
3.6.2 Flowchart Diagram:	25
3.6.3 Use Case Diagram:	28
3.6.4 Sequence Diagram	30
3.6.5 Class Diagram:	32

3.7	<i>Software Interface:</i>	34
3.8	<i>Security Requirements:</i>	36
3.9	<i>Project Development Tools:</i>	38
3.9.1	<i>Google Colab Pro:</i>	38
3.9.2	<i>Figma:</i>	39
3.9.3	<i>Canva (Inspiration):</i>	40
3.9.4	<i>Android Studio:</i>	41
3.9.5	<i>Flutter:</i>	43
3.9.6	<i>Dart:</i>	44
3.9.7	<i>Firebase:</i>	45
Chapter 4: IMPLEMENTATION AND METHODOLOGIES FOR ARTIFICIAL INTELLEGE		
PSYCHOTHERAPY:		48
4.1	<i>Mobile Application Screens:</i>	48
4.1.1	<i>Splash Screen:</i>	48
4.1.2	<i>Sign-up Screen:</i>	48
4.1.3	<i>Login Screen:</i>	51
4.1.4	<i>Forgot Password Screen:</i>	53
4.1.5	<i>Chat screen:</i>	54
4.2	<i>Implementation:</i>	55
4.2.1	<i>Large Language Model Selection:</i>	55
4.2.2	<i>Fine-Tuning Process:</i>	61
4.2.3	<i>Data Collection:</i>	64
4.2.4	<i>Data Generation:</i>	65
4.2.5	<i>Environment Setup:</i>	66
4.2.6	<i>Incomplete Implementation Objectives:</i>	67
Chapter 5: TESTING AND EVALUATION:		69
5.1	<i>Flutter Application Testing:</i>	69
5.2	<i>Large Language Model Testing:</i>	70
5.3	<i>Firebase Testing:</i>	74
5.4	<i>Flask Server Testing:</i>	75
5.5	<i>Integration Testing:</i>	76
5.6	<i>Benchmarking with Large Language Models:</i>	76
5.7	<i>Human Evaluation:</i>	77
5.8	<i>Combined Analysis:</i>	77

Chapter 6: BUSINESS MODEL	79
6.1 Key partners:	79
6.2 Key activities:	79
6.3 Key resources:	80
6.4 Value propositions:	80
6.5 Customer relationships:	81
6.6 Channels:	81
6.7 Customer segments:	82
6.8 Cost structure:	82
6.9 Revenue stream:	83
6.10 SWOT Analysis:	84
6.10.1 Strengths:	84
6.10.2 Weaknesses:	84
6.10.3 Opportunities:	84
6.10.4 Threats:	84
Chapter 7: CONCLUSION AND FUTURE WORK:	86
7.1 Conclusion:	86
7.2 Future Work:	86
REFERENCES	87
APPENDIX A: CODE OF THE BACKEND SERVER	89
APPENDIX B: CODE OF FIREBASE	91
APPENDIX C: CODE OF FINE TUNING LARGE LANGUAGE MODEL	93
APPENDIX D: CODE OF LOAD LARGE LANGUAGE MODEL	97

LIST OF FIGURES

Figure 2- 1 WYSA Mobile Application	13
Figure 2- 2 Woebot Mobile Application	14
Figure 2- 3 Replika Mobile Application	15
Figure 2- 4 Mindspa Mobile Application	16
Figure 2- 5 Microsoft Bing Mobile Application	17
Figure 3- 1 Architecture Diagram	22
Figure 3- 2 Flowchart Diagram	25
Figure 3- 3 Use Case Diagram	28
Figure 3- 4 Sequence Diagram	30
Figure 3- 5 Class Diagram.....	32
Figure 3- 6 Google Colab Pro IDE.....	38
Figure 3- 7 Figma Website	39
Figure 3- 8 Canva	40
Figure 3- 9 Android Studio Website	41
Figure 3- 10 Android Studio IDE	42
Figure 3- 11 Flutter Website.....	43
Figure 3- 12 Dart Website	44
Figure 3- 13 Firebase Website.....	45
Figure 4- 1 Splash Screen Page	48
Figure 4- 2 Sign-up Screen.....	49
Figure 4- 3 Login Screen	51
Figure 4- 4 forgot password Screen.....	53
Figure 4- 5 Chat Screen	54
Figure 4- 6 Falcon LLM	55
Figure 4- 7 Example responses from Bimedex LLM.....	57
Figure 4- 8 Encoder-decoder Transformer Architecture	59
Figure 4- 9 Sample of Dataset	64
Figure 5- 1 Training Loss VS Validation Loss	70
Figure 5- 2 Training Loss VS Learning Rate	70

LIST OF TABLES:

Table 4- 1 Comparison Between Falcon 7 – B and Falcon LLMs family	56
Table 4- 2 Sample of Generated Dataset	65

INTENDED AUDIENCE

This document is designed to cater to a diverse audience involved in the development, management, and utilization of the project. The intended readers include developers, project managers, marketing staff, end-users, testers, and documentation writers. Each of these roles plays a crucial part in the lifecycle of the project, and this document aims to provide the necessary information tailored to their specific needs.

- **Developers:**

- Developers are the primary audience for the technical sections of this document. They should focus on the detailed architecture, coding standards, API specifications, and integration guidelines to ensure they can effectively build and maintain the software.

- **Project Managers:**

- Project managers should begin with the executive summary and project overview to understand the project's scope, objectives, and timelines. They can then move on to the sections detailing project milestones, resource allocation, risk management, and deliverables.

- **Marketing Staff:**

- Marketing personnel will benefit from the sections that outline the unique features, target audience, and market positioning of the project. They should focus on the user personas, competitive analysis, and branding strategies to effectively promote the project.

- **End-Users:**

- Users should start with the introduction and overview sections to get a high-level understanding of the project. They should then review the user guides and feature descriptions to learn how to utilize the software effectively.

- **Testers:**

- Testers need to pay close attention to the testing methodologies, test cases, and quality assurance protocols described in the document. Detailed sections on bug tracking, performance testing, and usability testing will be particularly relevant to their roles.

- **Documentation Writers:**

- Those responsible for creating and maintaining documentation should start with the project overview to grasp the overall context. They should then delve into sections outlining the technical and user documentation requirements, ensuring they can produce clear and comprehensive guides for all user types.

READING SUGGESTIONS

To maximize the utility of this document, readers are encouraged to start with the overview sections. These sections provide a broad understanding of the project, its goals, and its structure. Once familiar with the general context, readers can then focus on the specific sections most relevant to their roles and responsibilities.

ABBREVIATIONS

AI	: Artificial Intelligence
API	: Application Programming Interface
BERT	: Bidirectional Encoder Representations from Transformers
CPU	: Central Processing Unit
FAQ	: Frequently Asked Questions
F1 Score	: F1-score
GPU	: Graphics Processing Unit
HTTPs	: Hypertext Transfer Protocol Secure
JSON	: JavaScript Object Notation
LLM	: Large Language Model
LSTM	: Long Short-Term Memory
ML	: Machine Learning
MLP	: Multi-Layer Perceptron
NLP	: Natural Language Processing
QA	: Question Answering
RAM	: Random Access Memory
ReLU	: Rectified Linear Unit
RNN	: Recurrent Neural Network
TFLite	: TensorFlow Lite
UI/UX	: User Interface/User Experience
YAML	: YAML Ain't Markup Language

Chapter 1: Introduction

Chapter 1: INTRODUCTION

1.1 Overview:

Psychological and mental disorders, such as depression and anxiety, are complex and challenging to diagnose. These disorders can manifest in individuals of any age, race, or gender, posing a growing concern worldwide. According to the World Health Organization, approximately 5% of the global adult population, or 280 million people, suffer from depression. Similarly, the National Alliance on Mental Illness (NAMI) reports that 1 in 5 adults in the United States have been diagnosed with mental health issues.

Since the onset of the COVID-19 pandemic in 2019, the importance of psychological well-being has become a critical aspect of daily life. Despite this increased awareness, many individuals still feel uncomfortable discussing their mental health with others and tend to keep their issues to themselves.

In this context, conversational bots, or chatbots, have emerged as non-traditional methods for therapy, providing a novel approach for individuals suffering from psychological illnesses. These chatbots offer a platform for people to express their thoughts and concerns, potentially alleviating the discomfort associated with discussing mental health issues face-to-face.

1.2 Motivation:

In the dynamic realm of Artificial Intelligence, a growing need for innovative solutions that elevate quality of life and streamline various processes has emerged. Psychotherapy, a field not immune to these advancements, stands to benefit significantly. Traditional psychotherapeutic methods encounter obstacles such as affordability, accessibility, and trust concerns. The impetus behind crafting an AI-driven psychotherapy mobile app is to systematically address these challenges, cultivating an environment conducive to efficient therapeutic interventions while seamlessly integrating cutting-edge technology.

1.3 Problem Statement:

- **Daily Exposure to Mental Health Problems.**
- **Limited individuals display the courage to pursue psychology or psychotherapy.**
- **Misconceptions about Psychotherapy:**
 - Some perceive psychotherapists as exclusive to severe cases, dubbing them "doctors of the insane."
 - Mental health issues are often not regarded on par with biological diseases.
- **Economic Barriers:**
 - Affordability challenges for some individuals regarding psychotherapy sessions.
- **Trust and Disclosure Challenges:**
 - Shyness and trust issues hinder many from sharing personal problems with therapists.
 - Difficulty entrusting therapists with confidential information.
- **Limited Availability:**
 - Psychotherapists are not accessible 24/7, impacting immediate support during critical moments.

1.4 Objectives:

- Realtime response psychotherapy chatbot.
- Cost saving solution.
- Trustworthy and user-friendly application.
- An easy way for people to open up about their problems with no need to go anywhere.
- And implementing AI technologies in creating a personalized and effective therapy experience.

1.5 Problem Complexity:

1. Affordability challenges in traditional psychotherapy:

- Traditional therapy can be expensive, limiting access for individuals with financial constraints.

2. Limited accessibility to conventional psychotherapeutic services:

- Geographical and logistical barriers may hinder individuals from accessing in-person therapy.

3. Elevated levels of fear and trust issues in seeking therapy:

- Stigma and trust concerns can discourage individuals from seeking help in a traditional therapeutic setting.

4. Integration of Artificial Intelligence into psychotherapy applications:

- The challenge lies in seamlessly incorporating AI to enhance therapeutic outcomes without compromising human touch.

5. Development of a user-friendly and effective mobile app:

- Ensuring the app is intuitive, engaging, and caters to diverse user needs is essential for widespread adoption.

6. Ensuring privacy and data security in AI-driven therapeutic interactions:

- Safeguarding sensitive personal information is critical to build and maintain user trust.

7. Addressing the multifaceted aspects of mental health through technology:

- The complexity of mental health requires a nuanced approach to ensure the app addresses diverse needs effectively.

8. Navigating ethical considerations in AI-based psychotherapeutic interventions:

- Balancing the benefits of AI with ethical guidelines to prevent misuse or unintended consequences is crucial.

9. Tailoring therapeutic interventions to individual needs:

- Customizing AI algorithms to accommodate diverse personalities and mental health conditions for effective support.

10. Measuring and ensuring the app's efficacy:

- Developing robust metrics to assess the app's impact on mental health outcomes and continuously refining based on user feedback.

1.6 Constraints:

1. Cost:

- **Implementation Costs:**

- Include the expenses associated with purchasing an online integrated development environment for fine-tuning and downloading the model (Google Collab Pro), software development, and installation.

- **Ongoing Maintenance Expenses:**

- Consider the costs related to routine maintenance, updates, and potential repairs. Factor in the need for technical support and any associated costs.

2. Usability and Ergonomics:

- **Mobile Application Design:**

- Invest in a user-friendly design for the mobile application, considering the needs and preferences of users.

- **User Training:**

- Provide adequate training resources to ensure that end-users understand how to use the system effectively.

3. Security:

- **Data Security:**

- Protect user data through secure storage, transmission, and handling. Comply with data protection laws and consider mechanisms for user authentication, validation and authorization.

4. Functionality:

- **Response generation Accuracy:**

- Ensure the accuracy and reliability of the model responses to avoid false information or instructions.

5. Maintainability:

- **Remote Diagnostics:**

- Incorporate features that allow for remote troubleshooting, reducing the need for physical intervention.

- **Modular Design:**

- Design the system with modular components to simplify maintenance tasks and facilitate future upgrades.

6. Network Connectivity:

- **Redundancy:**

- Implement redundancy measures to ensure continued functionality in the event of network outages.

- **Bandwidth Considerations:**

- Optimize data transmission to minimize the impact on the network bandwidth, especially in scenarios with many connected devices.

7. Scalability:

- **Software Scalability:**

- Design the system architecture to handle an increasing number of users. Ensure that software infrastructure can scale effectively.

- **Data Handling:**

- Consider how the system will handle a larger volume of data as it scales. Implement strategies for efficient data storage, retrieval, and analysis.

8. Dataset availability:

- Encountered significant challenges in finding psychotherapy-related datasets.
- Most available datasets are either scarce or contain a limited number of records
- Large companies, such as Altibbi, create comprehensive datasets but do not release them as open-source resources.
- This scarcity of open-source datasets poses a challenge for researchers and developers.
- The lack of robust and comprehensive data hinders effective training and evaluation of AI models in psychotherapy.

Chapter 2: Related Work

Chapter 2: RELATED WORK

2.1 Research Papers:

- ***“Evaluation of an Arabic chatbot based on extractive Question-Answering Transfer Learning and Language Transformers (August 2023)”***
 - This paper explores the current trends of using transformers and transfer learning techniques on Arabic chatbots.
 - It introduced new various Arabic transformers and semantic embedding models.
 - An illustration of Deep Learning models was provided, AraBERT, CAMeLBERT, AraElectraSQuAD, and AraElectra (Generator/Discriminator).
 - Two datasets were used for the evaluation between the models: one with 398 questions, and the other with 1395 questions and 365,568 documents sourced from Arabic Wikipedia.
 - It also magnifies the lens on practical applications in natural language processing tasks for Arabic chatbots that can make use of AraElectra-SQuAD which showed remarkable performance, high confidence, and robustness.

- ***“Artificial Intelligence Chatbot Behavior Change Model for Designing Artificial Intelligence Chatbots to Promote Physical Activity and a Healthy Diet (September 2020)”***

- The paper provides a comprehensive framework for the development of chatbot-based interventions to promote physical activity and healthy eating
- It focused on developing the AI chatbot’s core feature of natural language conversation to facilitate more flexible information exchange between humans and the chatbot.

- ***“IDDAT: An Ontology-Driven Decision Support System for Infectious Disease Diagnosis and Therapy (2018)”***

- The paper presents IDDAT, an ontology-driven decision support system for infectious disease diagnosis and antibiotic therapy.
- It addresses the limitations of existing decision support systems related to antibiotics, which mainly focus on therapy rather than diagnosis and are designed from a physician's perspective.
- IDDAT aims to identify infectious diseases based on patient-entered information and provide a specific antibiotic therapy adapted to the patient.
- The paper presents experimental results to evaluate the effectiveness of IDDAT.

- ***“A Predictive Text System for Medical Recommendations in Telemedicine: A Deep Learning Approach in the Arabic Context”***

- proposed a deep learning-based language generation model that simplifies the process of writing medical recommendations for doctors in an Arabic context.
- The proposed model was trained using data obtained from Altibbi databases related to medical recommendations, particularly gynecology, dermatology, psychiatric diseases, urology, and internist diseases.
- A Comprehensive comparison between different deep learning models was provided including Unidirectional and bidirectional long short-term memory (LSTM and BiLSTM), The one-dimensional convolutional neural network (CONV1D), and combination of LSTM and CONV1D (LSTM-CONV1D).
- The algorithms were trained using two versions of the datasets (i.e., 3-gram and 4-gram representations).

- ***“Deep learning for Arabic healthcare: Medical Bot (March 2023)”***

- Introduced the largest Arabic Healthcare Q&A dataset, called MAQA, consisting of over 430,000 questions distributed across 20 medical specializations.
- It was Proved that Transformer models can learn the training data, generalize to new data and has the lowest test loss, indicating that it is able to generalize to new data better than (LSTM and Bi-LSTM models).
- Transformer models are efficient to train.

2.2 Competitors:

2.2.1 “WYSA”:



Figure 2- 1 WYSA Mobile Application

- A mobile application made by Touchline Services, which is based in India. First released in 2015.
- Asks you questions to help you open about your problems.
- Stores your therapy sessions in a journal so that you can keep a check on your health.
- There are plenty of exercises to choose from, depending on what you're dealing with. Only some of those exercises are free.
- An SOS feature in case you find yourself in an emergency.

2.2.2 “Woebot”:

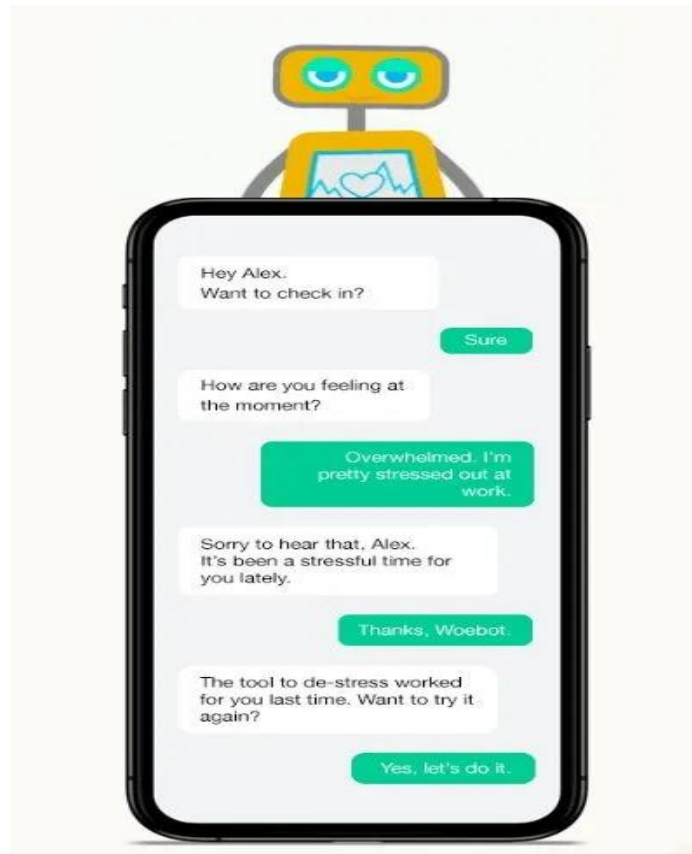


Figure 2- 2 Woebot Mobile Application

- A mobile application developed by psychologists and computer scientists from Stanford University. First released in the United States in 2017.
- let's you pick a time to add scheduled therapy sessions for each day.
- has a feature that allows you to select the common responses in the chat rather than typing the answer each time.
- It also offers targeted chat sessions to work on a specific issue.
- You can select your issue from the Topics tab. These sessions include interactive quizzes, tests, and games to help you overcome a challenge in a light-hearted way. In addition, the app provides an SOS feature in the Tools tab in case of emergencies.
- You can keep a journal with Woebot to track your progress.
- The app is completely free for everyone.

2.2.3 “Replika”:



Figure 2- 3 Replika Mobile Application

- A mobile application developed by Luka, Inc. of California, and it was first released in the United States in March 2017.
- You can select your issue from the Topics tab. The setup process is slightly long if you customize your Replika's avatar, but it's straightforward.
- Once you're done, it takes you straight to the chat. The chat itself is robust compared to the ones on the previous apps listed above, as you get to vote up or down on each response and can regenerate a response if you don't like it.
- Your companion also takes notes of important points and keeps them in memory. In addition, you can send voice notes, attach images, or video call your Replika.
- There's also an option to upgrade the chatbot to the pro version for better responses.

2.2.4 “Mindspa”:

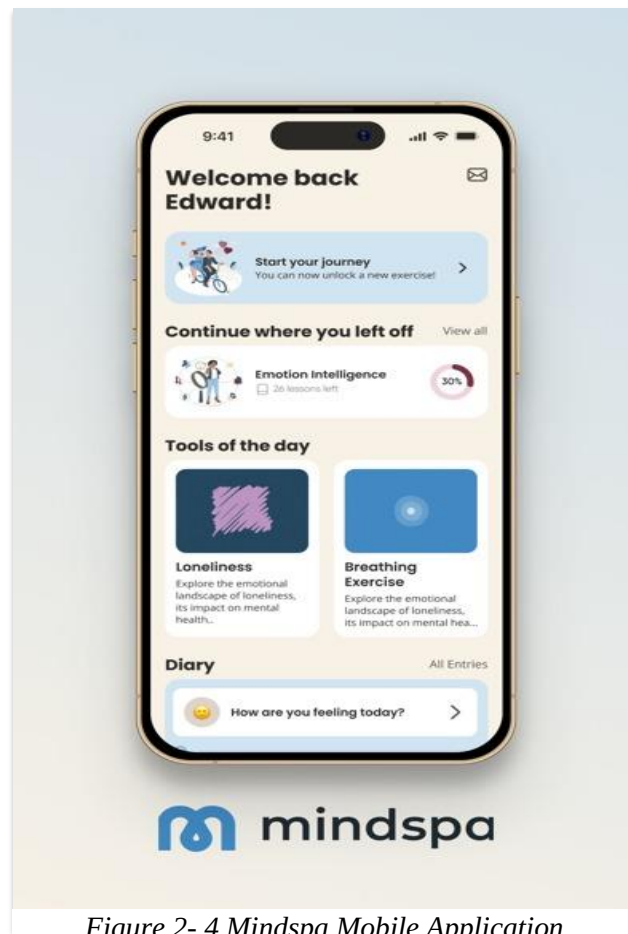


Figure 2- 4 Mindspa Mobile Application

- A mobile application developed in 2018.
- Mindspa is mostly a content-based app with articles, courses, and tasks on improving your mental health.
- The app also lets you keep a mental health diary.
- You have a conversation with the Mindspa chatbot mostly by selecting responses from a list.
- Most of the features this app offers are free.

2.2.5 “Microsoft Bing”:

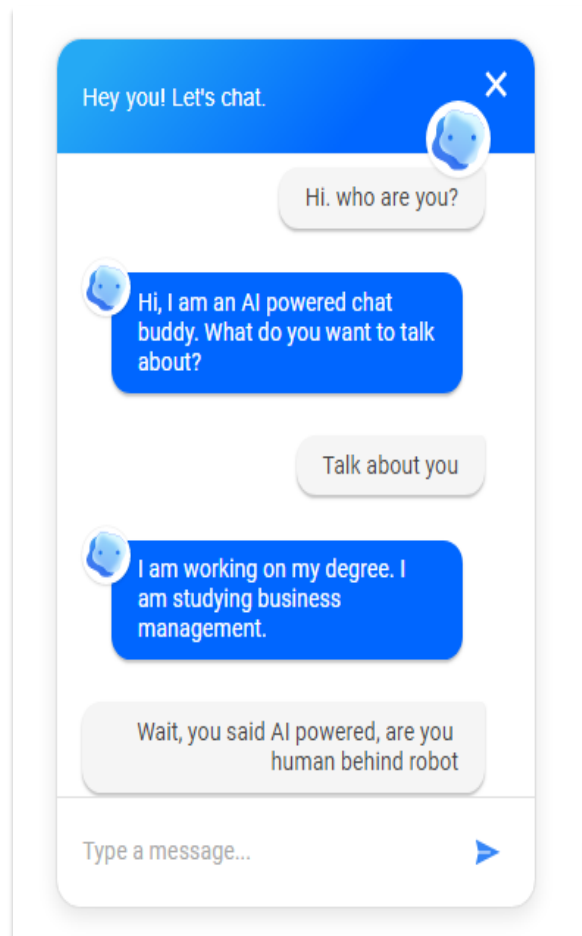


Figure 2- 5 Microsoft Bing Mobile Application

- The chatbot originally launched in a private preview in 2023, and Microsoft has been gradually opening it up ever since.
- You can convert ChatGPT, Bard, and other LLMs into chatbot therapists.
- “Act like my psychologist and therapist. Apply all the best practices in your responses to offer great therapy.”
- You can add more detail to the prompt to customize your therapist according to your needs.
- Once you send this message, Bing will confirm and ask you how it can help.

Chapter 3: System Analysis and Design

Chapter 3: SYSTEM ANALYSIS AND DESIGN:

3.1 Resource Requirements:

- A computer is required to implement the project. it's crucial or involving the frontend and backend.
- A Mobile Phone is required to run/open that application and display the AICO features.
- The Internet is needed so that the user can use the application as well as communicate with the other person.

3.2 Product Features:

- **Emotional Analysis and Support:**
 - The AI Therapy application employs natural language processing techniques to analyze the user's emotional tone and sentiment. It recognizes and validates emotions expressed by the user and provides empathetic and supportive responses. This feature aims to create a safe and understanding environment for users to express their emotions freely.
- **24/7 Availability:**
 - The AI Therapy application is available 24/7, allowing users to access therapy support at their convenience. Whether users have pressing concerns or simply need someone to talk to, the application ensures that therapy services are accessible whenever they are needed.
- **Secure and Confidential:**
 - The AI Therapy application prioritizes user privacy and data security. Robust measures are implemented to protect user information, adhere to applicable data protection regulations, and maintain confidentiality throughout the therapy process. Users can trust that their personal information and therapy sessions are kept confidential.
- **Integration with External Resources:**
 - The AI Therapy application can integrate with external resources and services to enhance its functionality.

3.3 Operating Environment:

- **Hardware Platform:**
 - The application can run on various devices, including smartphones, tablets, and computers. It is compatible with different operating systems, such as iOS and Android, ensuring broad accessibility for users. For the purposes of this project demonstration, we will be running it on the Android platform.
- **Operating System and Software Components:**
 - The application utilizes operating systems compatible with mobile devices. It leverages AI frameworks and libraries, such as Firebase for backend services and real-time database management, and utilizes Flask, a lightweight web framework, for developing and deploying web applications efficiently. Development tools like Android Studio IDE and Flutter are employed to build and enhance the mobile application. By incorporating these features and operating in a versatile environment, the AI Therapy application aims to provide users with a comprehensive and effective platform for mental health support.

3.4 Functional Features:

- The application shall allow users to create a new account by providing their username, email address, and password.
- The application shall enable users to sign in with their new account.
- The application shall allow users to send messages and chat with the app.

3.5 Non-functional Features:

- **Security:**
 - The application should ensure the privacy and security of user data, including personal health information, by implementing appropriate encryption methods, access controls, and regular security audits, and Adherence to healthcare data protection standards (e.g., HIPAA).

- **Compatibility:**
 - The application should be compatible with iOS and Android.
- **Availability:**
 - The application should be always accessible and available to users, except during maintenance windows.
- **Capacity:**
 - The application should be able to handle a specified number of users and requests simultaneously without degradation in performance.
- **Scalability:**
 - The application should be able to handle increased loads and usage without requiring significant changes or upgrades.
- **Usability:**
 - The application should be easy to use with a user-friendly interface and straight forward setup process.

3.6 Diagrams:

3.6.1 Architecture Diagram

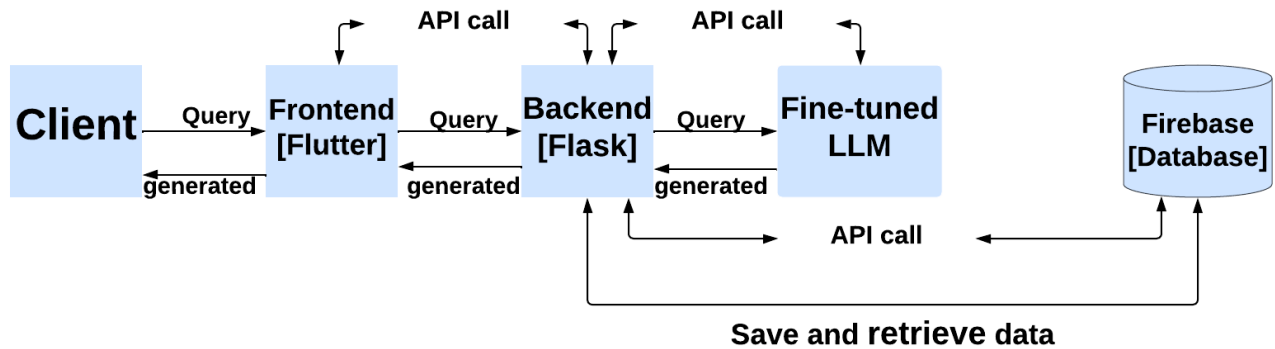


Figure 3- 1 Architecture Diagram

- **Client:**
 - The client represents the end-user interacting with the system. Users input their queries through the client interface. NoSQL.
- **Frontend [Flutter]:**
 - The frontend, developed using Flutter, serves as the intermediary between the client and the backend system.
 - Users' queries are sent from the client to the frontend, which then processes and forwards these queries to the backend.
 - Once responses are generated, the front end displays them back to the client.
- **Backend [Flask]:**
 - The backend is implemented using Flask, a lightweight web framework for Python.
 - The backend receives queries from the frontend and is responsible for further processing and routing them to the appropriate components.
 - After receiving responses from the LLM, the backend sends these responses back to the frontend.

- **Fine-Tuned LLM:**
 - The fine-tuned Large Language Model (LLM) which is Falcon-7B is the core AI component that processes queries and generates appropriate responses.
 - It interacts with the backend, receiving queries and sending back generated responses.
 - The LLM may also need to retrieve additional data from the Firebase database to provide contextually accurate responses

- **Database [Firebase]:**
 - The database, implemented using Firebase, stores all necessary data, including user queries and responses.
 - It serves two primary functions:
 - Data Retrieval:
 - The LLM queries the Firebase database to fetch relevant data needed to generate accurate responses.
 - Data Storage:
 - The system saves the query and corresponding response in the Firebase database for future reference and analysis.
 - The Firebase database ensures data persistence and supports the learning and improvement of the LLM over time.

- **Data Flow:**
 - **Query Flow:**
 - The process begins with the client generating a query. This query is sent to the frontend (Flutter), which then forwards it to the backend (Flask). The backend processes the query and sends it to the fine-tuned LLM.
 - **Response Generation:**
 - The LLM processes the query, possibly fetching additional data from the Firebase database, and generates a response.
 - **Response Flow:**
 - The generated response is sent back from the LLM to the backend, which then forwards it to the frontend. The front end displays the response to the client.
 - **Data Logging:**

- The system also ensures that each query and its corresponding response are saved in the Firebase database. This logging facilitates future data analysis and helps in refining the AI model.

3.6.2 Flowchart Diagram:

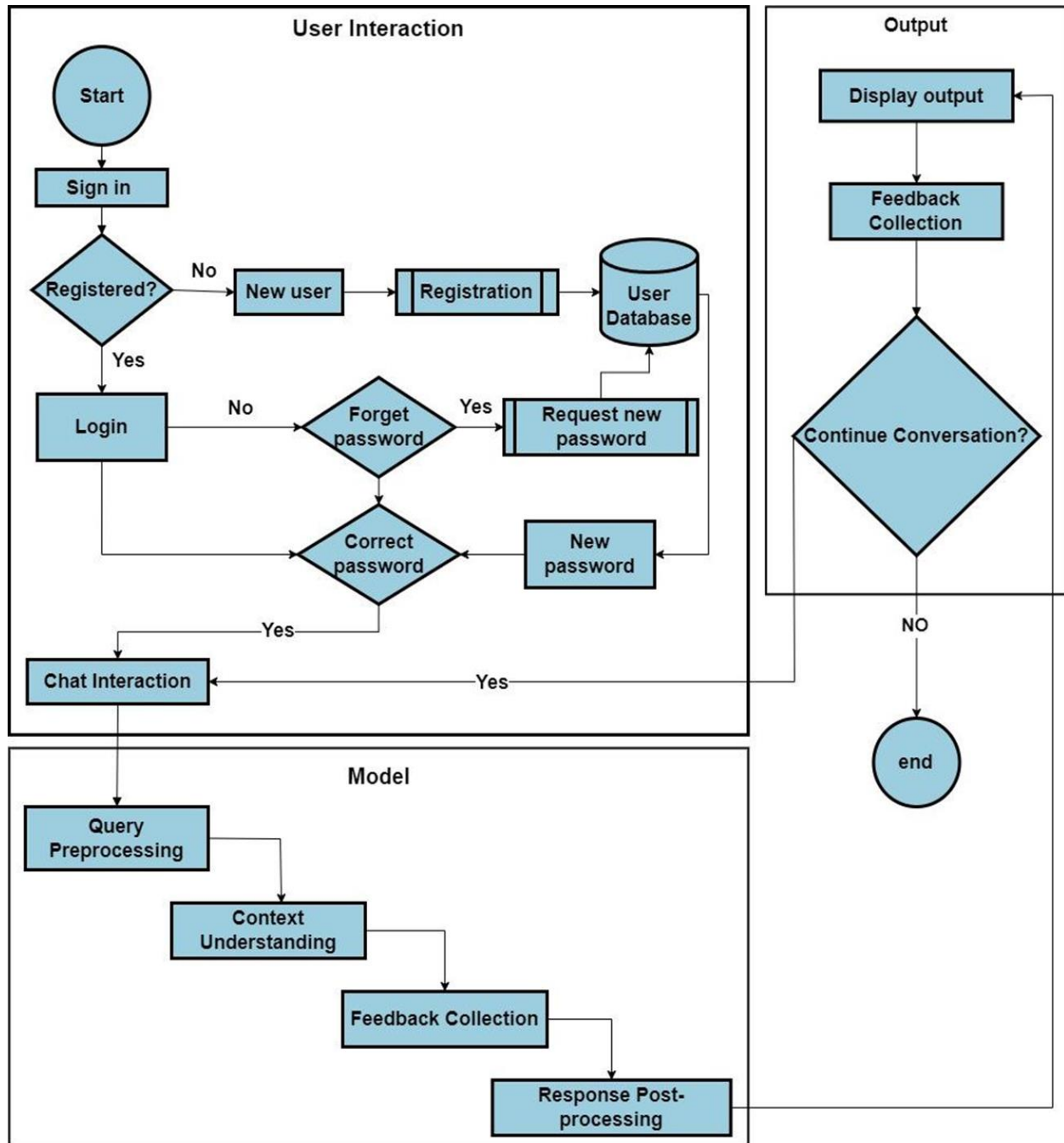


Figure 3- 2 Flowchart Diagram

- **Start:**
 - The interaction starts with the user initiating the process. This could involve opening a website or launching an application.
- **Sign In:**
 - The system prompts the user to sign in. This typically involves entering an email address and password associated with a registered account.
- **Registered User Check:**
 - The system checks if the user is registered by verifying their credentials against a database:
 - **Yes:** If the user is registered, the flowchart proceeds to the login process.
 - **No:** If the user isn't registered, they are directed to a new user registration section.
- **Login:**
 - The user enters their login credentials, typically an email address and password.
- **Login Verification:**
 - The system verifies the login credentials against the database.
 - **Correct Password:**
 - If the password is correct, the user is granted access to the system, and the flowchart proceeds to a section labeled "Chat Interaction." This could be a live chat window or an interface where the user interacts with the system.
 - **Incorrect Password:**
 - **Forget Password:**
 - If the user forgets their password, they can request a new password. This would likely involve entering their email address to receive a password reset link.
 - **New Password:**
 - The user can enter a new password directly. This scenario is less common but could be applicable in certain systems.

- **Chat Interaction:**
 - This section depicts an interaction between the user and the system, possibly through a chat interface. The user submits a query, which is then preprocessed, meaning the system gets the query ready for further processing.
- **Context Understanding:**
 - The system tries to understand the context of the user's query. This likely involves using natural language processing techniques to interpret the meaning of the user's input.
- **Model:**
 - The system utilizes a model, which could be a machine learning model or a set of rules, to respond to the user's query.
- **Response Post-Processing:**
 - The system refines the response generated by the model before delivering it to the user.
- **Feedback Collection:**
 - The system might collect feedback from the user about the interaction. This feedback could be used to improve the system's performance in the future.
- **End:**
 - The interaction ends after the feedback collection stage.

3.6.3 Use Case Diagram:

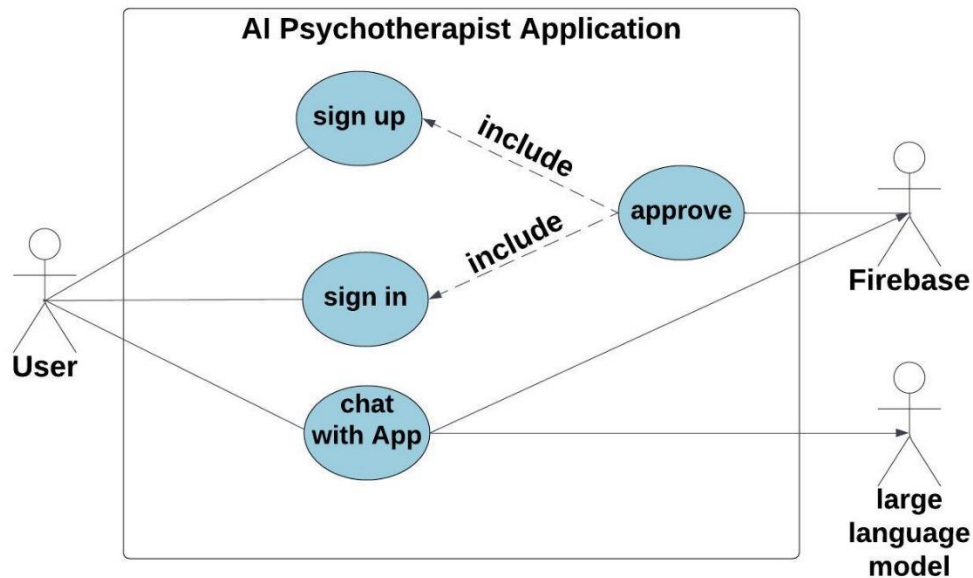


Figure 3- 3 Use Case Diagram

- **Actors:**

- User: The primary actor in the system. This represents the individuals who will interact with your mobile application in order to access the psychotherapist chatbot.

- **Use Cases (System Functionalities):**

- **Sign Up:**

- This use case describes the process by which a new user creates an account within the mobile application.

- **Sign In:**

- This use case covers the user entering their existing credentials to log in and gain access to the application's functionalities.

- **Chat with App:**

- This use case represents the core functionality where the user interacts with the application through a chat interface, conversing with the Large Language Model (LLM) acting as the psychotherapist chatbot.

- **System Boundary:**

The system boundary is depicted by a rectangle encompassing the mobile application (including the LLM chatbot) and the functionalities it offers.

- **Interactions:**

- The user interacts with the system by initiating the three use cases: Sign Up, Sign In, and Chat with App.
- A dotted line from the "Chat with App" use case connects to a box labeled "Firebase." This illustrates that the chat conversation history is saved in the Firebase database.

3.6.4 Sequence Diagram

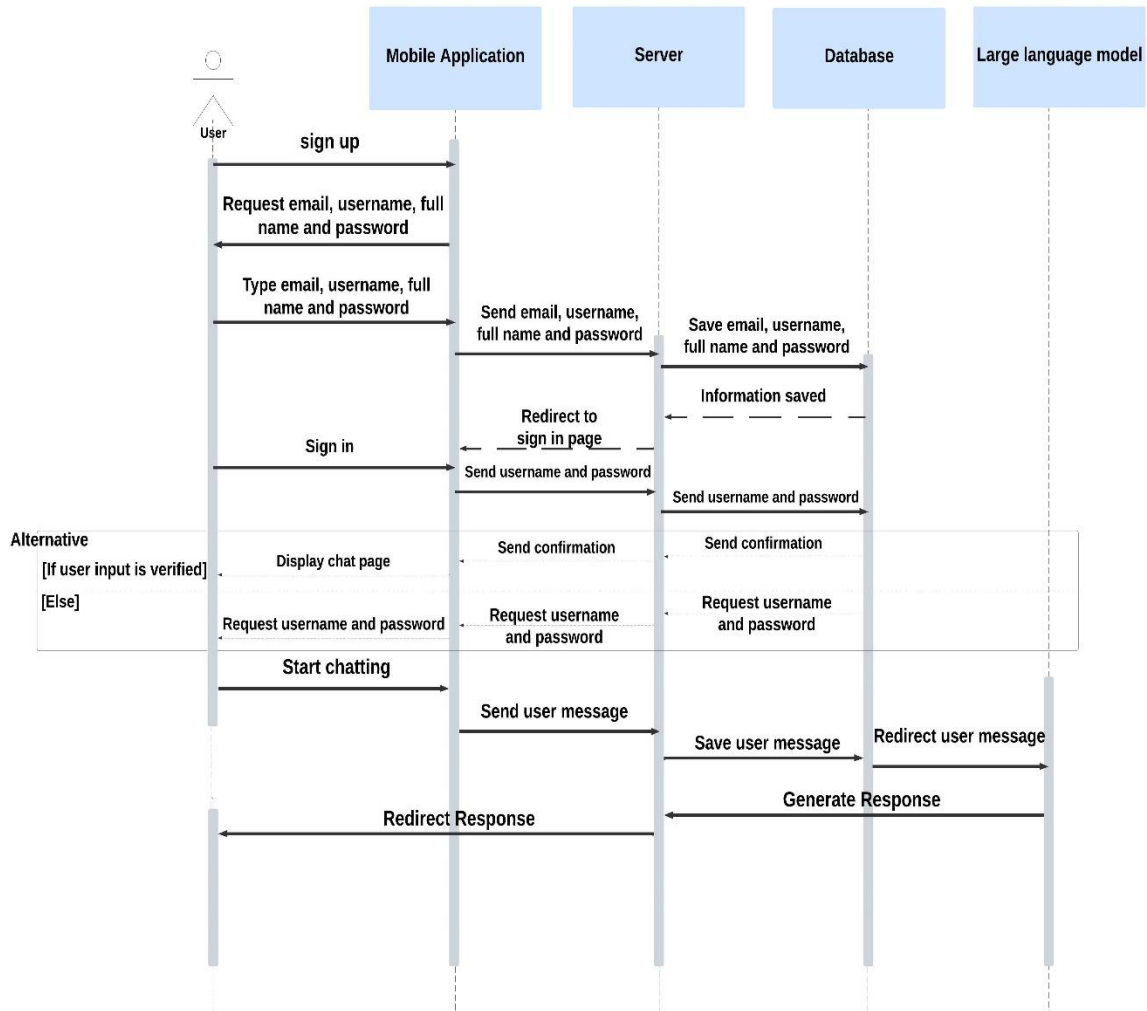


Figure 3- 4 Sequence Diagram

- **Mobile App:**
 - This represents the mobile application itself, including the user interface and functionalities for signing up, signing in, and chatting with the LLM chatbot.
- **Server (Firebase):**
 - This represents the Firebase backend that handles user authentication and stores the chat history.
- **User:**
 - This represents the user of the mobile application.

- **Sign Up:**

- The user interacts with the mobile app by entering their email address, username, and password. The mobile app sends this information (sign-up request) to the server (Firebase).
- The server validates the user information and creates a new user account in the Firebase database.
- The server sends a confirmation message back to the mobile app.
- The mobile app redirects the user to sign-in page.

- **Sign In:**

- The user enters their username and password in the mobile app.
- The mobile app sends this information (sign-in request) to the server (Firebase).
- The server verifies the username and password against the Firebase user database.
- If the credentials are valid, the server sends a confirmation message back to the mobile app.
- If the credentials are invalid, the server sends an error message back to the mobile app.
- The mobile app displays either redirect user to chat screen or display error message to the user, depending on the server's response.

- **Chat with App:**

- The user interacts with the chat interface in the mobile app, typing their message.
- The mobile app sends the user's message to the server (Firebase).
- The server (Firebase) does not process the message itself (as it acts as a database). It might temporarily store the message or route it elsewhere for processing by the LLM chatbot.
- Processing by LLM chatbot: This part isn't explicitly shown in the diagram, but it's assumed that the message is routed to the LLM chatbot for processing. The LLM chatbot generates a response, which is then routed back to the server.
- The server (Firebase) receives the LLM chatbot's response and sends it back to the mobile app.
- The mobile app displays the LLM chatbot's response to the user in the chat interface.

3.6.5 Class Diagram:

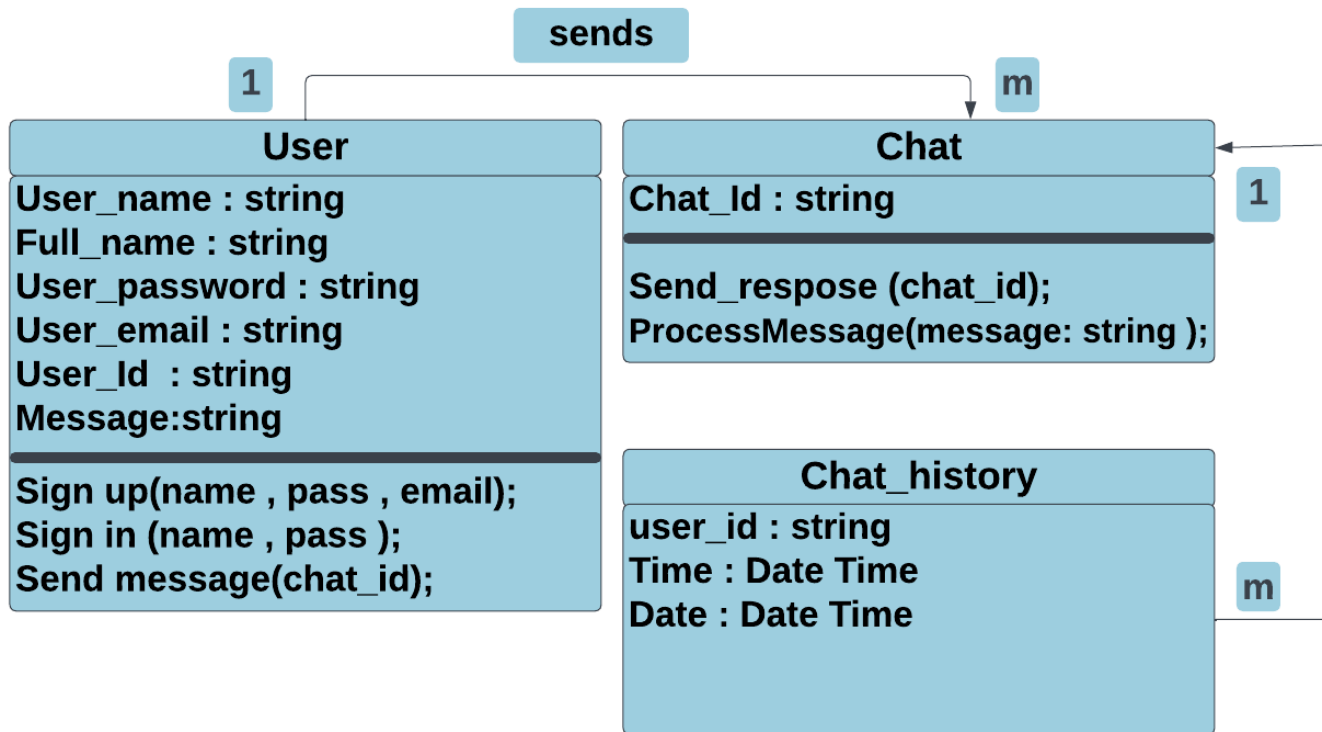


Figure 3- 5 Class Diagram

- **User:**

- This class represents a user of the mobile application. It likely contains attributes such as:
 - username (string) - for identification within the app
 - email (string) - used for signup and potentially password reset
 - password (string) - hashed for security purposes
 - unique user ID (string) - a unique identifier within the system

- **Chat:**

- This class represents a chat session or conversation between the user and the LLM chatbot. It likely contains attributes such as:
 - **chat ID (string):** a unique identifier for the chat session
 - **timestamps (date/time):** recording the start and potentially end time of the chat
 - **user ID (string):** a reference to the User object participating in the chat
 - **list of chat messages (optional):** a collection of ChatMessage objects associated with the chat session (potentially represented by another class)

- **ChatHistory:**

- This class represents the user's chat history, potentially containing a collection of Chat objects:
 - **user ID (string):** a reference to the User object
 - **list of chats (Chat class):** an ordered collection of Chat objects representing the user's chat history

- **Relationships:**

- **User:** One-to-Many relationship with Chat. A single user can have many chat sessions.
- **Chat:** Many-to-One relationship with User. A chat session belongs to one user.
- **ChatHistory:** One-to-One relationship with User (optional). A user has one chat history.
- **ChatHistory:** Aggregation relationship with Chat (optional). The ChatHistory class holds a collection of Chat objects representing the user's chat history.

3.7 Software Interface:

- **User Interfaces:**

- The GUI follows a clean and modern design, ensuring ease of navigation. Sample screen images are available in the user interface specification document. GUI standards adhere to industry best practices for optimal user experience.
- Screen Layout Constraints:
 - Consistent layout across screens for familiarity and ease of use.
 - Standard buttons (e.g., Sign up, Login, forgot password, Back) maintain a uniform appearance.
 - Error Message Display Standards: Clear and concise error messages following a standard format. User guidance provided for error resolution.
 - Screen images and layout conform to mobile design guidelines.

- **External APIs:**

- **External API Interactions:**
 - The system is designed to interact with various external APIs to extend its functionalities. These interactions enable the integration of third-party services and features, ensuring that the application can leverage external resources to enhance user experience and operational capabilities.
- **API Access and Compatibility:**
 - The system ensures continuous access and maintains compatibility with external API specifications. This involves regularly updating and adapting the system to comply with any changes in the API endpoints, authentication methods, and data formats, thereby guaranteeing seamless and uninterrupted integration.

- **Database Management:**

- The system utilizes Firebase for managing all database interactions, providing robust and efficient handling of data. This includes seamless retrieval and storage of information during user sign-in and sign-up processes, as well as maintaining comprehensive logs of chat sessions. The integration with Firebase ensures real-time data synchronization across all devices, and secure management of user credentials and communication logs, thereby enhancing the overall reliability and security of the application.

- **Data Definition and Messaging:**

- The system meticulously defines data items and messages to facilitate efficient data sharing between various system components. This includes specifying the structure, format, and protocol for data exchange, ensuring that all components can communicate effectively, and that data integrity is maintained throughout the system.

3.8 Security Requirements:

- **User Authentication and Authorization:**
 - **Strong Password Policy:**
 - Implement a robust password policy that requires a combination of Capital and small letters, numbers, and special characters.
 - **Role-Based Access Control (RBAC):**
 - Implement RBAC to ensure that users have access only to the resources necessary for their role. This minimizes the risk of unauthorized access to sensitive data.
- **Data Encryption:**
 - **Data-at-Rest Encryption:**
 - Ensure that all sensitive data stored in the database is encrypted using strong encryption algorithms provided by Firebase.
 - **Data-in-Transit Encryption:**
 - Use TLS (Transport Layer Security) to encrypt data transmitted between the client and server, ensuring that data remains confidential and secure during transmission.
- **Secure API Communication:**
 - **Authentication:**
 - Require API calls to be authenticated using secure tokens.
 - **Input Validation:**
 - Validate all inputs to APIs to prevent injection attacks and ensure data integrity.
- **Database Security:**
 - **Access Controls:**
 - Restrict database access to authorized personnel and applications only.
 - **Regular Audits:**
 - Conduct regular security audits and vulnerability assessments of the database to identify and address potential security risks.

- **Application Security:**
 - **Code Reviews:**
 - Perform regular code reviews to identify and mitigate security vulnerabilities.
Use automated tools to scan for common security issues.
 - **Secure Coding Practices:**
 - Follow secure coding practices and guidelines to minimize the risk of introducing vulnerabilities during development.
 - **Third-Party Libraries:**
 - Regularly update third-party libraries and dependencies to their latest versions to protect against known vulnerabilities.
- **Incident Response:**
 - **Incident Detection:**
 - Implement monitoring and logging mechanisms to detect and respond to security incidents in real-time.
- **User Data Privacy:**
 - **Data Minimization:**
 - Collect only the data necessary for the functionality of the application and avoid storing sensitive information unless absolutely required.
 - **User Consent:**
 - Ensure that users provide explicit consent for the collection and use of their data. Provide clear privacy policies and options for users to manage their data preferences.
 - **Data Anonymization:**
 - Anonymize personal data wherever possible to protect user privacy and comply with data protection regulations.
- **Compliance and Regulations:**
 - **Regulatory Compliance:**
 - Ensure that the application complies with relevant data protection regulations and industry standards.

3.9 Project Development Tools:

3.9.1 Google Colab Pro:

Settings

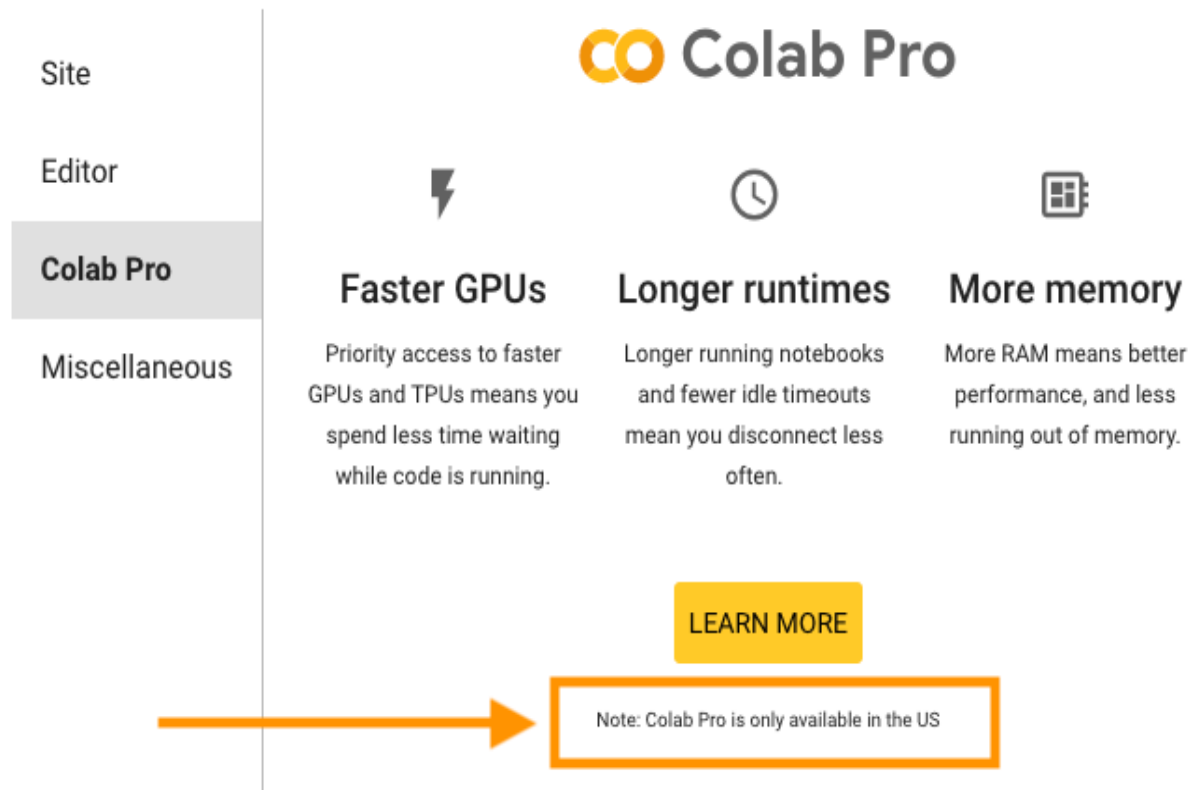


Figure 3- 6 Google Colab Pro IDE

- **Google Colab Pro** is a cloud-based platform that provides virtual machines with powerful GPUs (Graphics Processing Units) and RAM (Random Access Memory) for running computationally intensive tasks like machine learning. It's a paid upgrade to the free Google Colab service.
- **Our Usage:** We utilized Google Colab Pro's superior processing power to handle the demanding task of fine-tuning our large language model (LLM). The increased GPU and RAM capacity facilitated the efficient training process required for this project.

3.9.2 Figma:

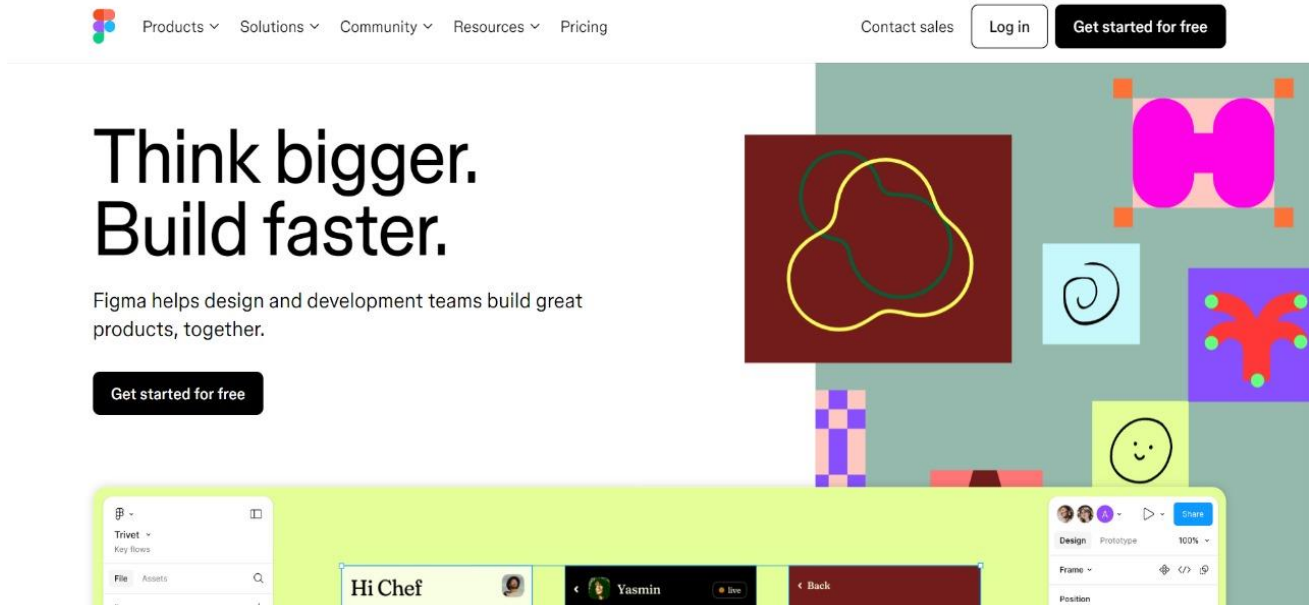


Figure 3- 7 Figma Website

- **Figma** is a web-based design tool that allows for collaborative creation and iteration of user interfaces (UI) and user experiences (UX). Designers can use Figma to prototype and visualize the look and feel of an application.
- **Our Usage:** We employed Figma to design the front-end or UI of our mobile application. Figma's collaborative features and intuitive interface facilitated the creation of high-fidelity mockups for all the different screens within the application.

3.9.3 Canva (Inspiration):

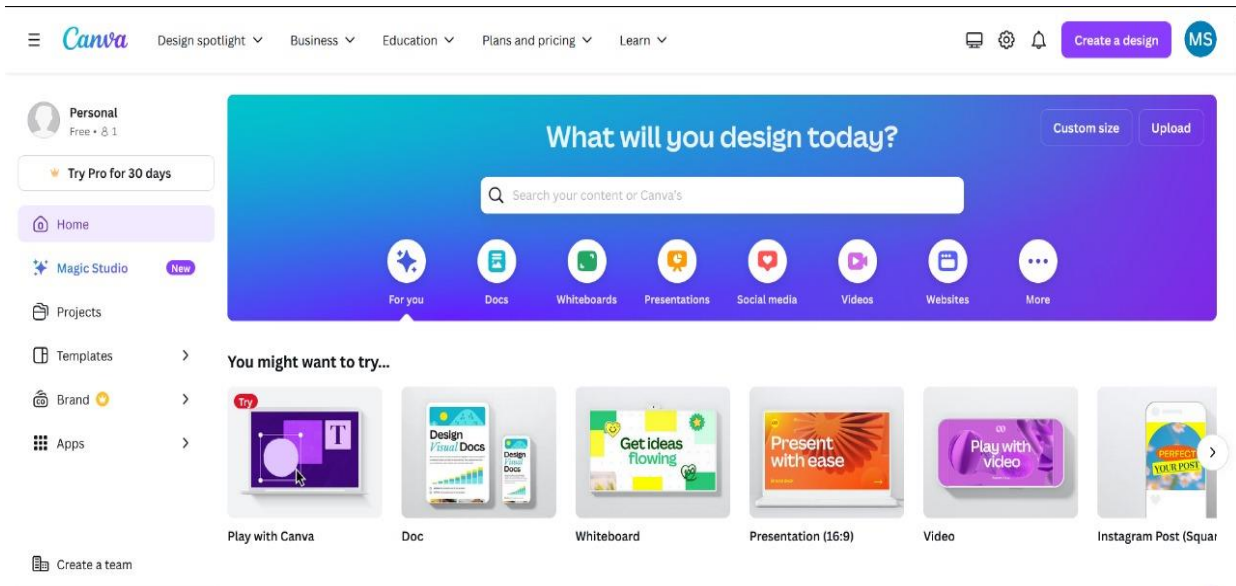


Figure 3- 8 Canva

- **Canva** is a graphic design platform that provides users with pre-designed templates and tools for creating social media graphics, presentations, and other visual content. It offers a user-friendly interface and a vast library of assets.
- **Our Usage:** While not directly used for development, Canva served as a source of inspiration for our front-end design. We leveraged Canva's templates and design elements to spark ideas and guide the overall aesthetic of the mobile application's UI.

3.9.4 Android Studio:

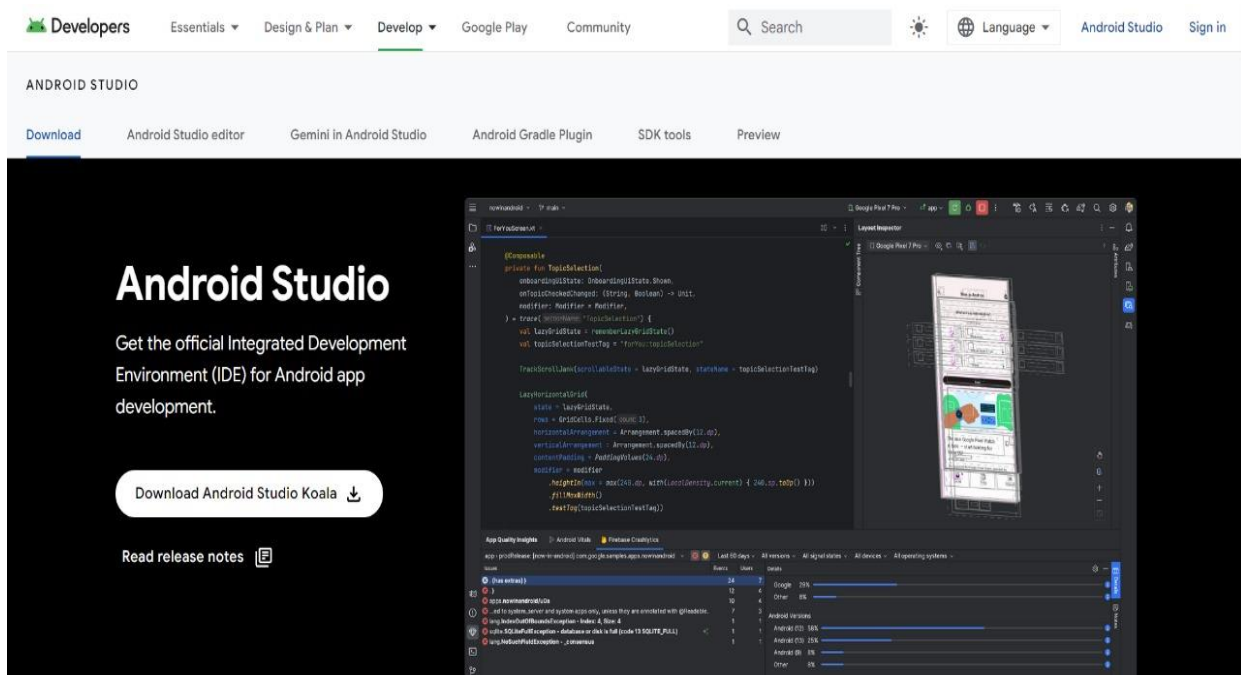


Figure 3- 9 Android Studio Website

- **Android Studio** is the official Integrated Development Environment (IDE) created by Google for building Android applications. It provides a comprehensive set of tools for development, testing, and debugging, including an emulator for simulating the Android operating system.

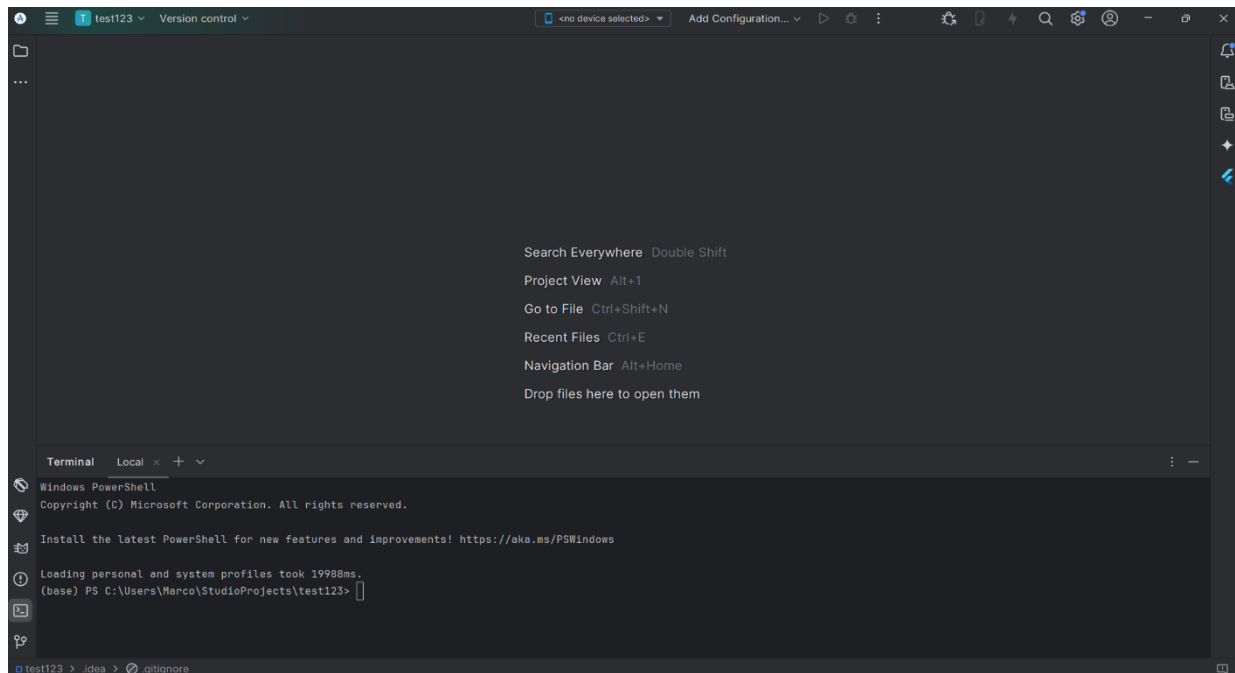


Figure 3- 10 Android Studio IDE

- **Our Usage:** We used Android Studio as the primary development environment for creating our mobile application. We utilized its built-in features and functionalities to write code, manage project resources, and leverage the Android emulator to test and debug the application on a simulated Android device.

3.9.5 Flutter:

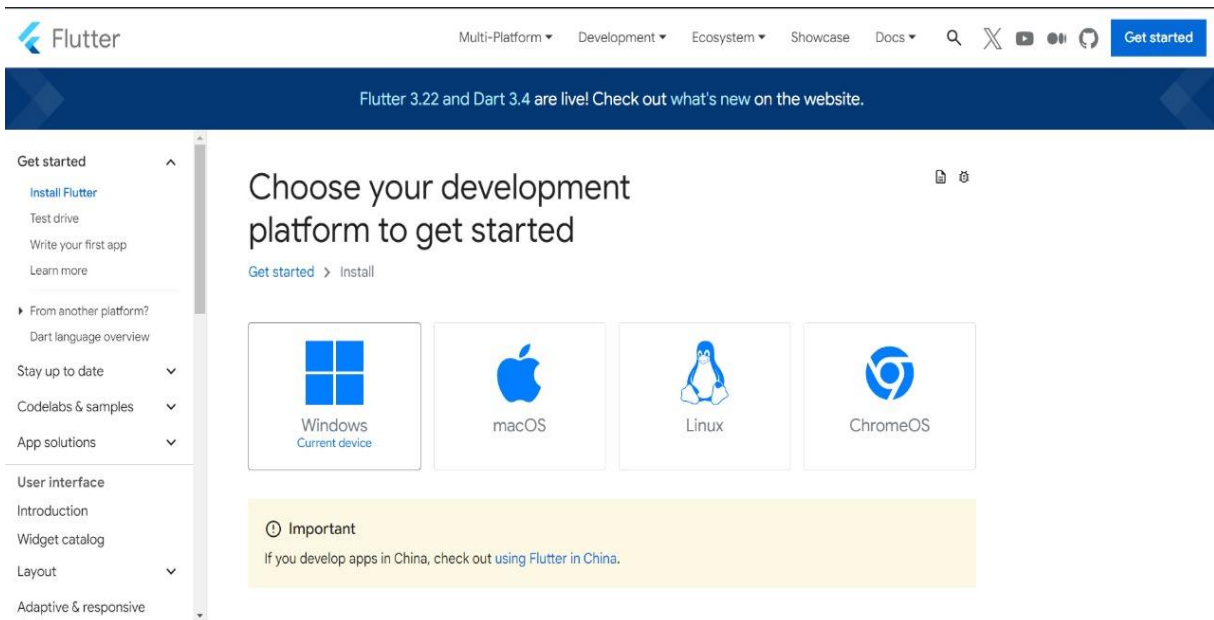


Figure 3- 11 Flutter Website

Flutter is an open-source framework developed by Google for building beautiful, cross-platform mobile applications using the Dart programming language. We chose Flutter due to its ability to create apps that work seamlessly on both Android and iOS platforms, saving development time and resources.

3.9.6 Dart:

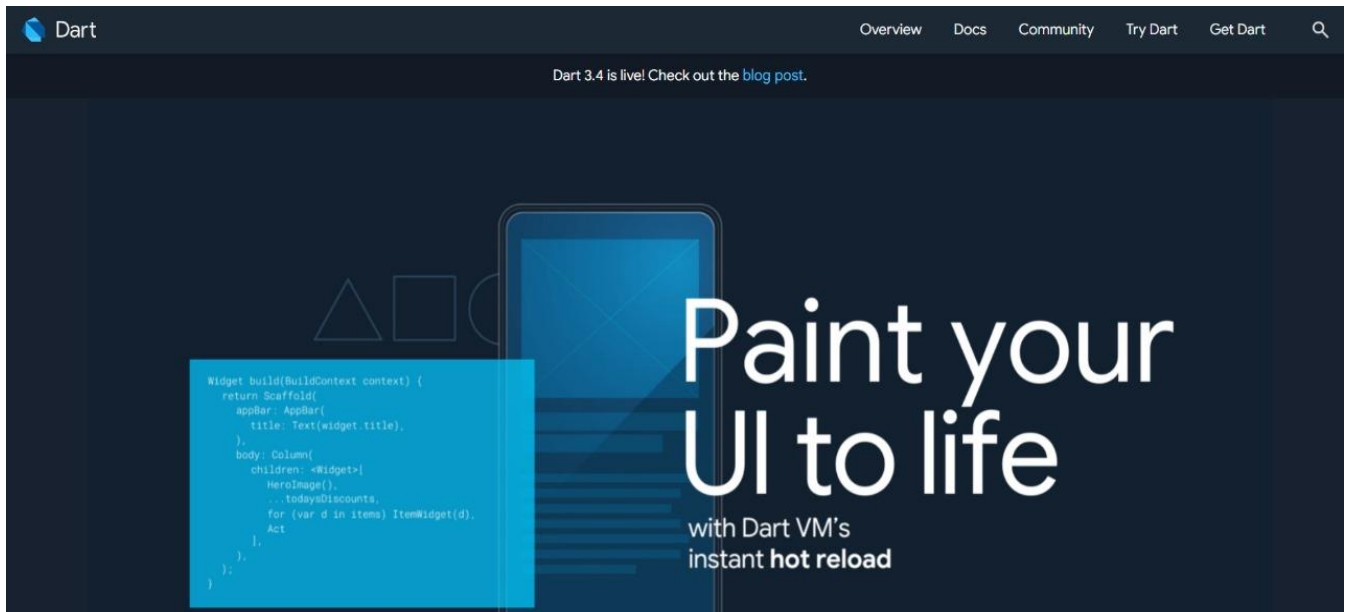


Figure 3- 12 Dart Website

Dart is an object-oriented, general-purpose programming language created by Google specifically for building modern web and mobile applications. It's known for its readability, speed, and ability to compile into native code for different platforms. We opted for Dart due to its compatibility with Flutter and its suitability for creating the core functionalities of our mobile application.

3.9.7 Firebase:

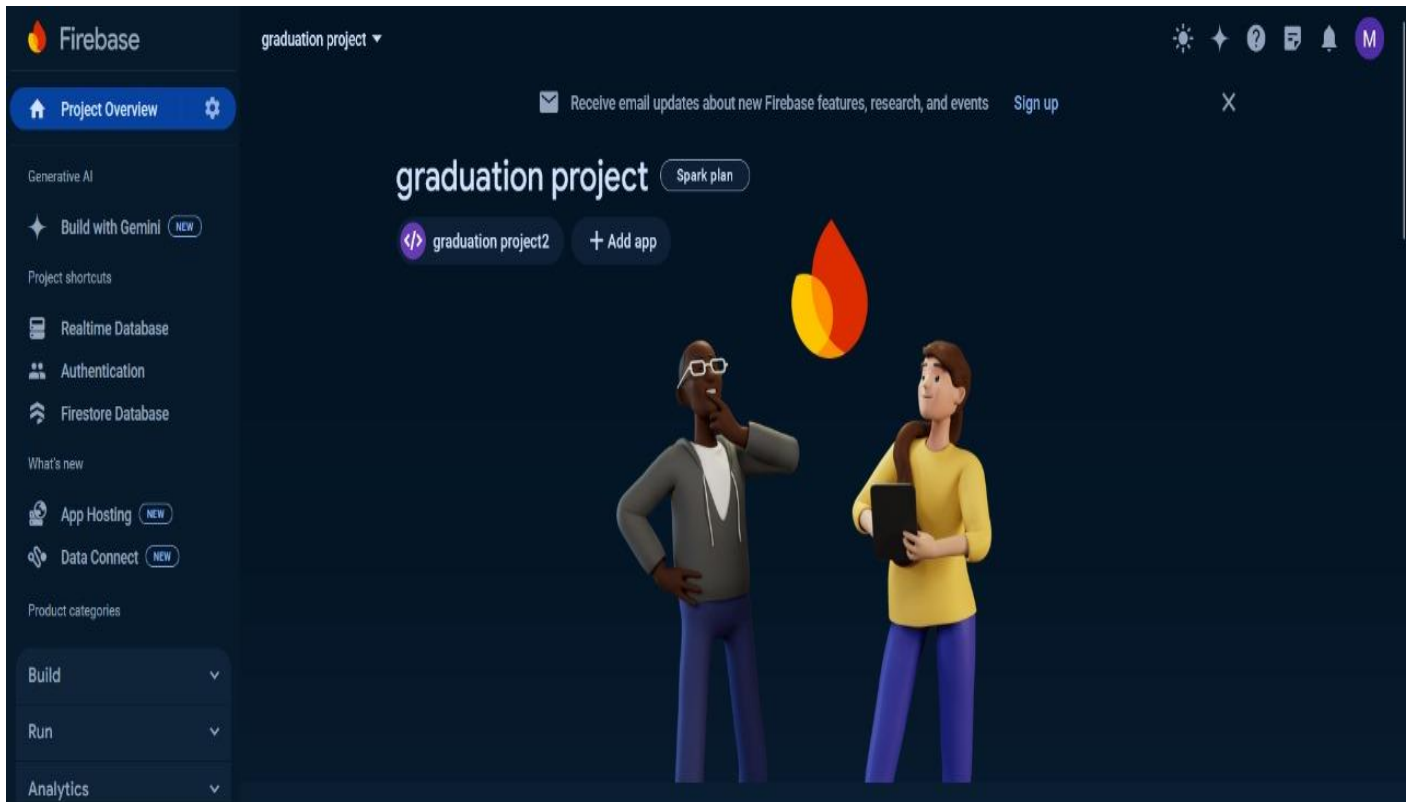


Figure 3- 13 Firebase Website

- **Firebase** is a Google-backed mobile application development platform that provides a suite of tools and services to simplify the backend development process.
- Our Usage: We employed Firebase as our primary backend solution. It offered functionalities like:
 - **Database Management:**
 - A flexible and scalable NoSQL database was used to store various types of data for our application, including user profiles, model responses, and the complete chat history between the user and the LLM.
 - **Authentication & Security:**
 - Firebase's robust authentication services were leveraged to manage user accounts within the application. This included features like user registration, login, validation, and security measures to ensure data integrity and protection.

In essence, this combination of tools provided a comprehensive development environment for our project. Google Colab Pro facilitated the fine-tuning of the LLM, design tools like Figma and Canva aided in crafting the user interface, and Android Studio with Flutter and Dart served as the foundation for building the mobile application. Finally, Firebase provided the robust backend infrastructure to manage user data, authentication, and other critical functionalities.

Chapter 4: Implementation And Methodologies

Chapter 4: IMPLEMENTATION AND METHODOLOGIES FOR ARTIFICIAL INTELLIGENCE PSYCHOTHERAPY:

4.1 Mobile Application Screens:

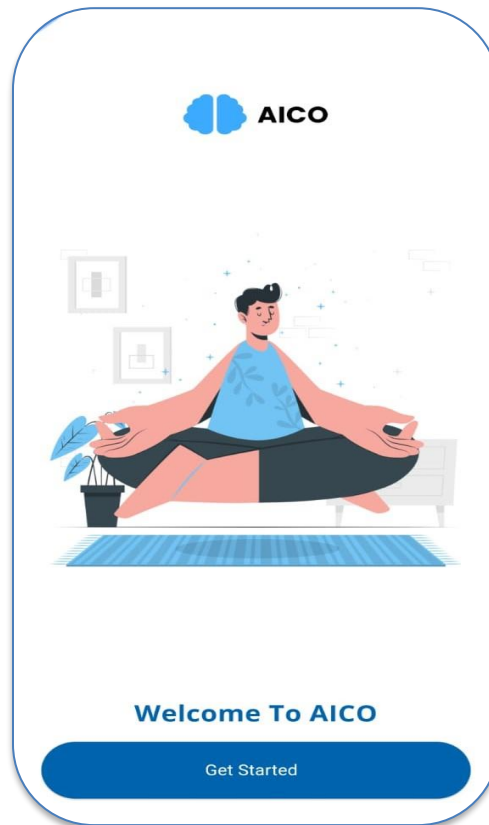


Figure 4- 1 Splash Screen Page

4.1.1 Splash Screen:

- The AICO app opens with a visually appealing splash screen that sets the tone for the user experience. The screen features the centered AICO logo. Below the logo is the text "Welcome to AICO" in a clear and easy-to-read font.
- The bottom portion of the splash screen displays a call to action with the phrase "Get Started" written in a button style. This invites users to begin their journey with the AICO app.
- Overall, the splash screen is simple, informative, and inviting. It effectively communicates the app's brand identity while also guiding users towards their first interaction within the app.

4.1.2 Sign-up Screen:

Welcome!

Enter Full name

Enter Username

Enter E-mail

Enter password

re_enter password

Sign Up

Already have an account? [Log In](#)

DESIGNED BY 12 PIXELS

Figure 4- 2 Sign-up Screen

- The sign-up screen prompts users for several pieces of information to create a new account. Here's a breakdown of the fields:
 - **Enter Full Name:**
 - This field likely requires the user to enter their actual first and last name.
 - **Enter Username:**
 - This field is where the user creates a username or handle that will be used to identify them within the app. Usernames should be unique and chosen carefully as they will be visible to other users.
 - **Enter E-mail:**
 - The user enters their email address, which will be used to create their account and for any future communication.
 - **Enter Password:**
 - This field is where the user creates a strong password to secure their account. Here are some tips for creating a strong password:
 - Includes at least eight characters.

- Use a combination of upper and lowercase letters, numbers, and symbols.
 - Avoid using dictionary words or personal information like birthdays or addresses.
- **Re-enter Password:**
 - The user re-enters their password in this field to confirm that it was typed correctly.
- **Important Note:**
 - All the fields on the sign-up screen must be filled out before the user can proceed.
- **Strong Password:**
 - The signup screen doesn't explicitly mention password requirements

4.1.3 Login Screen:

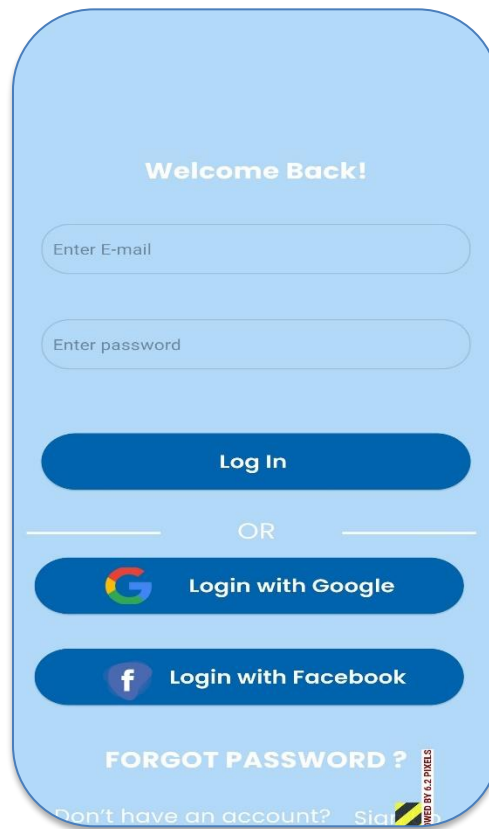


Figure 4- 3 Login Screen

- **The Login screen prompts users for two pieces of information to log in:**
 - **Enter Email:**
 - This field is likely where the user enters the email address associated with their account.
 - **Enter Password:**
 - This field is where the user enters the password they created when they signed up for the account.
- **The login screen also offers three alternative ways:**
 - **Login with Google:**
 - This button allows users to log in to the app using their Google account credentials.

- **Login with Facebook:**
 - This button allows users to log in to the app using their Facebook account credentials.
- **Forgot Password:**
 - This link leads to a page where users can request a new password if they have forgotten their current one.

4.1.4 *Forgot Password Screen:*

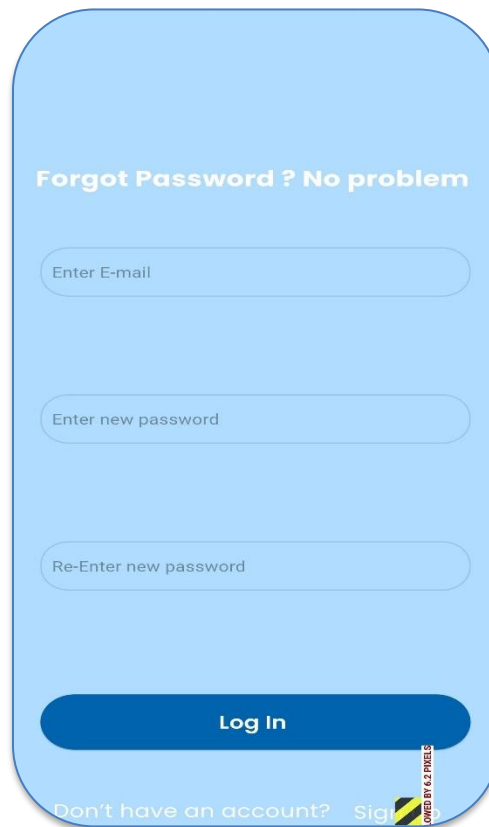


Figure 4- 4 forgot password Screen

- The forgot password screen offers a way to recover a forgotten password. Here's what the screen prompts users to enter:
 - **Enter E-mail:**
 - This field requires the user to enter the email address associated with their account. The system will then send a password reset link to that email address.
 - **Log In:**
 - This button directs users back to the login screen.
- The screen also offers an alternative way to create a new account:
 - **Sign Up:**
 - This link directs users to the sign-up screen where they can create a new account.

4.1.5 Chat screen:

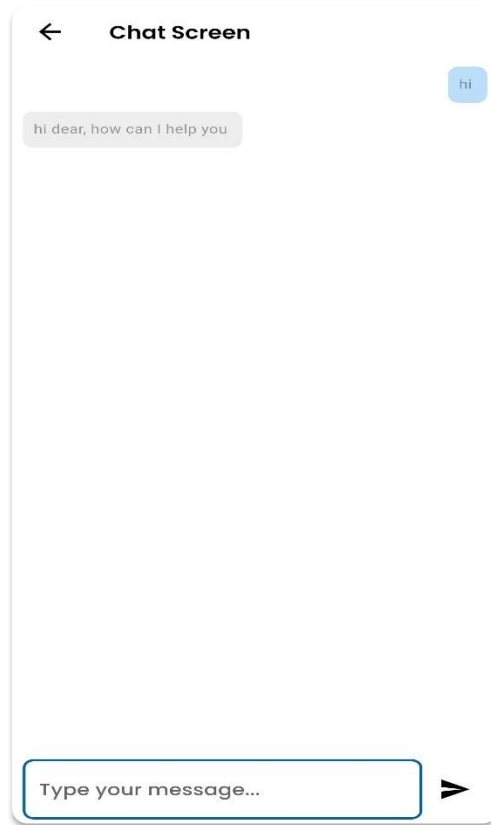
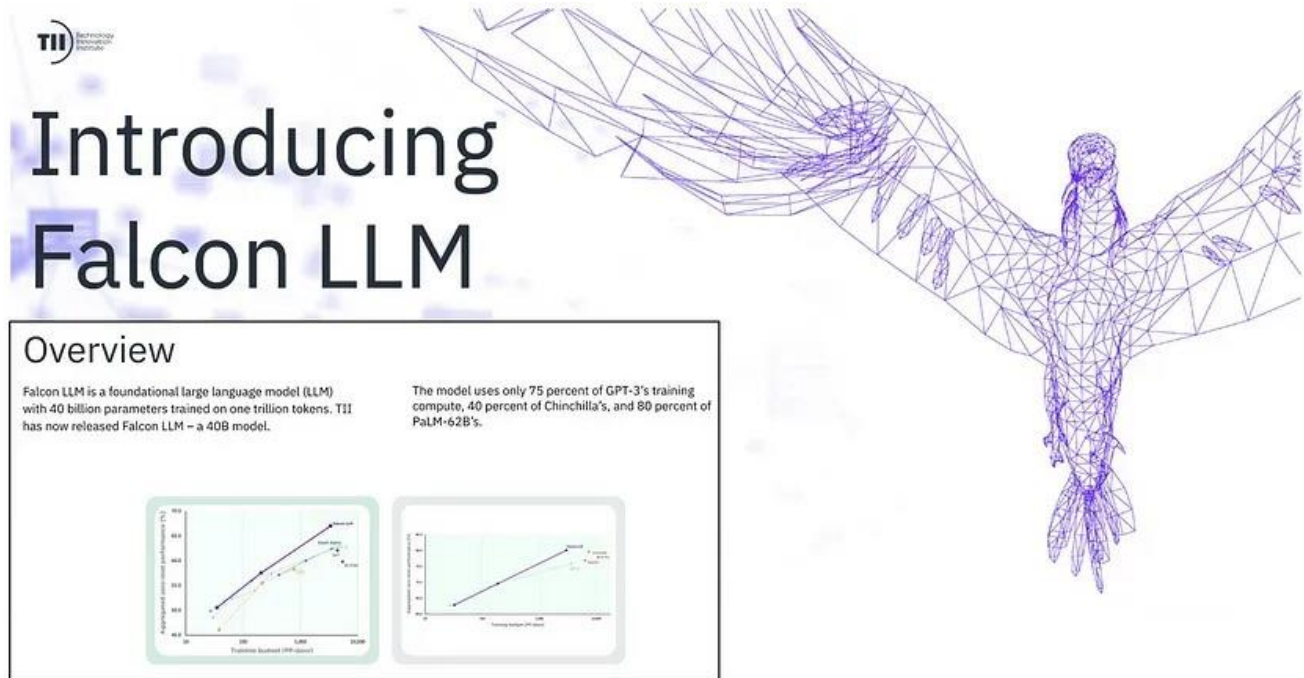


Figure 4- 5 Chat Screen

- The chat screen depicts a basic interaction between the user and the large language model:
 - **Initiator:**
 - The chat begins with a message from the chatbot, suggesting it starts the conversation.
 - **User Input:**
 - The "Send Message" button signifies a designated area where the user can type and send their response to the chatbot.
 - **Functionality:**
 - This initial message from the chatbot aims to prompt the user to begin their inquiry or request.

4.2 Implementation:

4.2.1 Large Language Model Selection:



Falcon LLM (Large Language Model) is a series of advanced AI language models designed to perform a wide range of natural language processing tasks. These models are part of a broader movement in artificial intelligence that leverages deep learning to understand, generate, and interact with human language more effectively. Falcon LLMs are particularly notable for their scalability, versatility, and the extensive pre-training they undergo, which allows them to excel in various applications from text generation to language translation and more.

- **Evaluation between Falcon LLMs family:**

Feature/Model	Falcon-1B	Falcon-7B	Falcon-40B	Falcon-180B
Parameters	1 billion	7 billion	40 billion	180 billion
Training Data	Moderate	Large	Very Large	Extremely Large
Performance	Basic	High	Very High	State-of-the-Art
Inference Speed	Very High	High	Moderate	Lower (due to size)
Cost	Low	Moderate	High	Very High
Deployment Complexity	Low	Moderate	High	Very High
Use Case Suitability	Simple NLP tasks	Intermediate to advanced NLP tasks	Advanced and complex NLP tasks	Highly complex and nuanced NLP tasks
Resource Requirements	Low	Moderate	High	Very High

Table 4- 1 Comparison Between Falcon 7 – B and Falcon LLMs family

- **Balancing Power on Laptops:**

- **Challenge:** Powerful LLMs (like Falcon-40B & 180B) boast impressive parameters but require specialized hardware (servers/TPUs) that laptops lack.
- **Solution:** Falcon-7B strikes a balance with 7 billion parameters, excelling on laptops due to its efficient design

- **Architectural Efficiency for Laptops:**

- **Beyond Parameters:**
 - The underlying architecture also matters. Falcon-7B utilizes a streamlined causal decoder architecture, unlike larger models.
 - Why it Matters: This structure excels at tasks requiring sequential processing (common for laptops) and efficiently utilizes laptop resources.

- **Evaluation between Falcon 7-B model and other LLMs:**



Figure 4- 7 Example responses from Bimedex LLM

- **Reduced Hardware Requirements:**
 - Compared to other large models with much higher parameter counts, Falcon-7B avoids the need for expensive or inaccessible hardware.
- **Improved Efficiency for Sequential Tasks:**
 - Unlike models with full encoder-decoder structures, the streamlined architecture of Falcon-7B avoids unnecessary computational overhead, leading to better performance on a laptop.
- **Enhanced Project-Specific Performance:**
 - Open-source access allows for customization that closed-source models lack, potentially leading to better performance on your specific project tasks.
- **Enhanced Project Support:**
 - An active community can offer valuable assistance beyond the model itself, leading to a smoother development process on your laptop project.
- We explored Bimedex LLM, a potentially promising option. However, its significant size (Approx. 200 GB) presented a mismatch.

- ***The Falcon-7B Architecture:***

While the Falcon-7B model boasts an impressive "causal decoder-only" architecture, delving into its intricate details presents a unique challenge. This section explores the reasons behind the limited public knowledge about architecture and proposes alternative approaches for a comprehensive understanding within the context of this book.

- ***Limited Public Disclosure:***

The Technology Innovation Institute (TII), the developers behind the Falcon family of LLMs, has opted to withhold detailed information regarding the full architecture of these models. This strategic move likely stems from the protection of intellectual property. Just as any innovative creation requires safeguarding, the intricate design choices and functionalities within the Falcon models are considered proprietary information by TII.

- ***Website Focus:***

The official Falcon LLM website prioritizes showcasing the capabilities and user benefits these models offer to potential users. In-depth technical specifications take a backseat to highlighting the practical applications and functionalities of the Falcon family. This user-centric approach ensures potential users understand the value proposition of these models without getting bogged down in the technical complexities.

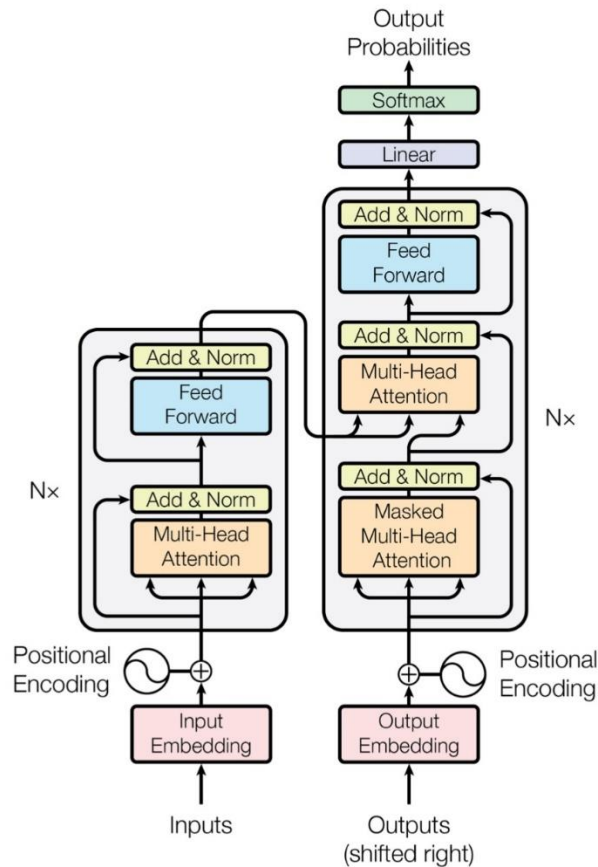


Figure 4- 8 Encoder-decoder Transformer Architecture

- **General Architecture of a Causal Decoder-Only:**

- The illustrated Transformer architecture highlights its core functionality: the encoder-decoder structure.
- This structure divides the model into two distinct parts:
 - The encoder analyzes and understands the input sequence.
 - The decoder uses this encoded information to generate the output sequence.
- The Falcon-7B model deviates from this traditional approach.
- Falcon-7B employs a simplified strategy by using only the decoder's capabilities for its language processing tasks.
- This suggests that the Falcon-7B model might rely on pre-trained knowledge or embeddings to represent the input, bypassing the explicit encoding stage.

- **Architecture Components:**

- **Embedding Layer:**

- This layer takes the input text (or code) and converts it into numerical representations called embeddings. These embeddings capture the meaning and relationships between words (or tokens) in the input sequence.

- **Decoder Layers (multiple):**

- These are the workhorses of the model. Each layer takes the encoded information from the previous layer and processes it further.

- **Self-Attention Mechanism:**

- This mechanism allows the model to focus on relevant parts of the input sequence when generating the output. However, in a causal decoder, this attention is "masked" to prevent the model from peeking ahead in the sequence. It can only attend to information it has already processed.

- **Positional Encoding:**

- Since the model doesn't see the entire sequence at once, positional encoding helps it understand the order of the elements in the input.

- **Feed Forward Network:**

- This network adds non-linearity to the model, allowing it to learn more complex relationships within the data.

- **Output Layer:**

- This layer takes the final output from the decoder layers and generates the final output sequence, which can be text, code, or another desired format.

4.2.2 *Fine-Tuning Process:*

- ***Pre-Trained Foundation:***

- We begin by utilizing a pre-trained Large Language Model (LLM) that has been trained on a massive dataset. This LLM possesses a strong understanding of language structure and can generate generic responses.

- ***Targeted Training:***

- Instead of training an entirely new model from scratch, we leverage the pre-trained LLM as a starting point. When using a pre-trained model, lower layers often capture generic features, while higher layers capture more specific features related to the original training task. Freezing the lower layers allows you to retain these generic features while only fine-tuning the higher layers to adapt to the new task.

- ***Domain-Specific Adaptation:***

- We train these higher layers on a new, focused dataset specifically curated for the psychotherapist chatbot application. This dataset likely includes examples of therapy conversations, allowing the model to learn the nuances and language patterns relevant to this domain.

- **Challenges:**

- **Overfitting:**

- **Poor Generalization:** An overfitted model captures the noise and random fluctuations in the training data rather than the underlying patterns. As a result, it fails to generalize to new data, leading to poor performance on validation and test sets.
 - **High Variance:** Overfitting typically results in a model with high variance, meaning its predictions fluctuate significantly depending on the specific training data it was given.
 - **Unreliable Predictions:** An overfitted model can make unreliable and inconsistent predictions when exposed to new data, making it less useful in real-world applications.
 - **Increased Complexity:** Overfitting often results in overly complex models that are harder to interpret and maintain.

- **Underfitting:**

- **Inaccurate Predictions:** An underfitted model fails to capture the relationships in the data, leading to consistently inaccurate predictions.
 - **Bias:** Underfitting is often associated with high bias, where the model makes strong assumptions about the data and oversimplifies the true complexity of the relationships.
 - **Missed Insights:** By not capturing the underlying patterns, an underfitted model misses valuable insights that could be critical for decision-making and understanding the data.
 - **Inefficiency:** Training an underfitted model is inefficient, as it doesn't make full use of the available data to improve its performance.

- ***The Advantages of Fine-Tuning:***

- **Efficiency:**

- By leveraging the pre-trained model's existing knowledge, fine-tuning significantly reduces training time and computational resources compared to building a model from scratch. This is particularly beneficial for projects with limited datasets or computational power.

- **Enhanced Performance:**

- Fine-tuning allows the model to specialize in a specific domain, in this case, psychotherapist chat interactions. The focused training on relevant data improves the model's ability to understand and respond effectively within this context.

- **Faster Convergence:**

- Since the pre-trained model has already learned useful features, fine-tuning typically requires fewer epochs to achieve good performance. This results in faster convergence during training.

- **Reduced Training Time and Resources**

- Fine-tuning can significantly reduce the amount of time and computational resources required for training. This is because only the higher layers or a subset of the model's parameters need to be updated, rather than training all layers from scratch.

4.2.3 Data Collection:

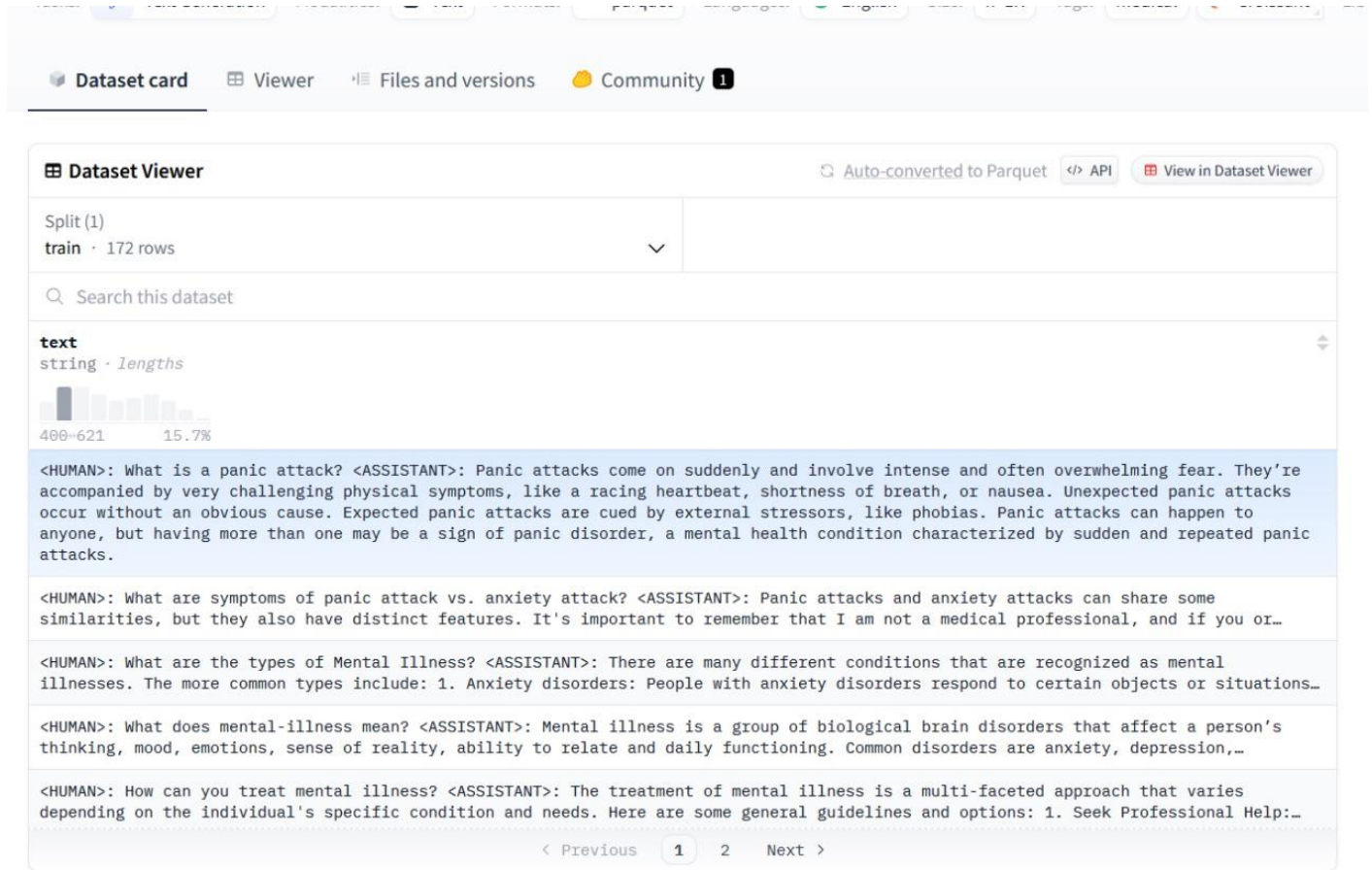


Figure 4- 9 Sample of Dataset

We initially attempted to leverage a dataset from **Kaggle**, a popular repository for machine learning data. This data set contained approximately **172 records** related to psychotherapy called “Mental health support” [6]. However, a crucial challenge emerged – the data lacked the essential question-and-answer format crucial for effective training. While the dataset offered valuable insights into doctor suggestions, it lacked representation of patient problems and their conversational style. Unfortunately, data scraping techniques to obtain patient-centric data proved infeasible due to the unavailability of suitable online resources.

4.2.4 Data Generation:

	Question	Answer
1	I feel hopeless and can't find joy in anything. I barely have the energy to get out of bed.	I'm really sorry you're feeling this way. It sounds like you're experiencing severe depression. I'd like to start you on an antidepressant called sertraline. We'll begin with a low dose and adjust it as needed. Alongside this, I'd recommend starting cognitive-behavioral therapy (CBT) to help you develop coping
2	I can't concentrate on work, and I feel constantly fatigued.	It seems like you're struggling with significant symptoms of depression, which can indeed affect concentration and energy levels. I'll prescribe fluoxetine to help improve your mood and concentration. Additionally, therapy can provide techniques to help manage fatigue and improve focus. Try setting small,
3	I have lost interest in all the activities I used to enjoy.	Losing interest in activities is a core symptom of depression. We'll start you on venlafaxine, an SNRI that can help boost your mood and interest in daily activities. In therapy, we'll work on gradually reintroducing activities that you used to enjoy. Also, try to engage in physical activities like yoga or swimming, which can naturally
4	I feel worthless and have thoughts of self-harm.	These thoughts are very concerning, and I'm glad you shared them with me. We'll start you on escitalopram to help stabilize your mood. I also want to connect you with a crisis intervention therapist immediately to provide additional support and develop a safety plan. Engaging in mindfulness practices like meditation can
5	My sleep is disrupted; I either sleep too much or too little.	Sleep disturbances are common in depression. Mirtazapine can help regulate your sleep patterns while also improving your mood. We'll also discuss sleep hygiene practices like maintaining a regular sleep schedule, avoiding caffeine before bed, and creating a restful sleeping environment.

Table 4- 2 Sample of Generated Dataset

To address the data scarcity issue, we adopted an innovative approach. We employed a different LLM to generate a new dataset specifically tailored to our needs. This generated dataset comprised nearly **1200 records**, make split for Training, Validation and Testing in a question-and-answer format, simulating a therapeutic conversation.

4.2.5 *Environment Setup:*

Optimizing the training environment was crucial for successful fine-tuning of the LLM. We prioritized utilizing a Graphics Processing Unit (GPU) over a Central Processing Unit (CPU) due to the significant performance boost GPUs offer for computationally intensive tasks like LLM training. Additionally, we recognized the need for increased RAM capacity to handle the data processing demands.

- **Server Exploration:**

Initially, we attempted to leverage a server provided by our University, The Arab Academy for Science, Technology and Maritime Transport, but the server needed maintenance and wasn't available at the current time.

Also using an online service provider like AWS or Microsoft Azure was a choice but running the application over the cloud wasn't a purpose in this project.

- **Local PC as a Last Resort:**

Given the time constraints and limitations with the academy server, we resorted to utilizing a local PC for training the LLM. While not ideal from a performance standpoint, this approach allowed us to complete the essential training phase within the project timeframe.

4.2.6 *Incomplete Implementation Objectives:*

While we successfully achieved core project milestones, time constraints necessitated the postponement of certain implementation goals:

- **High-Performance Server Deployment:**
 - Ideally, the LLM would benefit from training and deployment on a powerful server with significant computational resources. This would ensure optimal performance and scalability for future iterations of the project.
- **Android Device Integration:**
 - Our ultimate vision involved a native Android application where the trained LLM would be loaded and accessible directly on user devices. This integration, leveraging TensorFlow Lite for mobile deployment, was unfortunately deferred due to time constraints.

Chapter 5: Testing And Evaluation

Chapter 5: TESTING AND EVALUATION:

5.1 *Flutter Application Testing:*

- **Unit Testing:**
 - We employed unit testing frameworks like Dart unit testing to isolate and test individual functionalities of the Flutter codebase. This ensured proper behavior of UI elements, data manipulation, and user interaction logic within the mobile application.
- **Widget Testing:**
 - We utilized widget testing capabilities within Flutter to verify the visual appearance and behavior of individual UI components under various conditions. This ensured a consistent and visually appealing user interface across different screen sizes and devices.
- **Integration Testing:**
 - Following individual component testing, we conducted integration testing to verify how different parts of the Flutter application interact with each other. This ensured seamless data flow and communication between UI elements and backend services.

5.2 Large Language Model Testing:

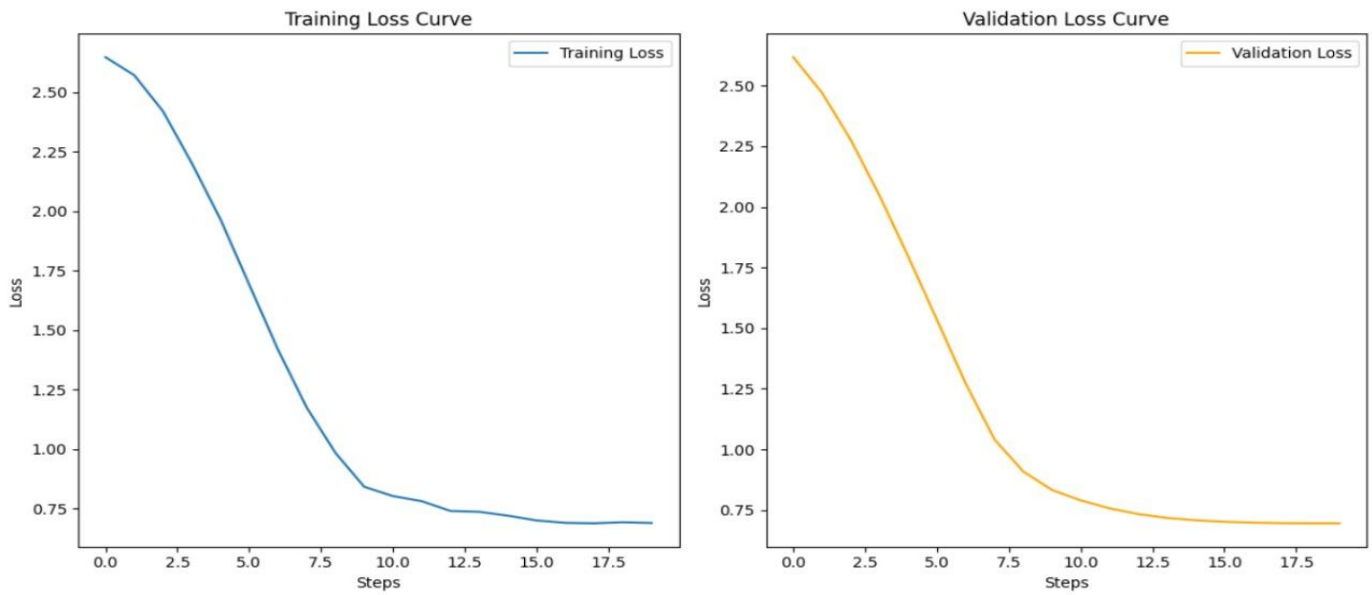


Figure 5- 1 Training Loss VS Validation Loss

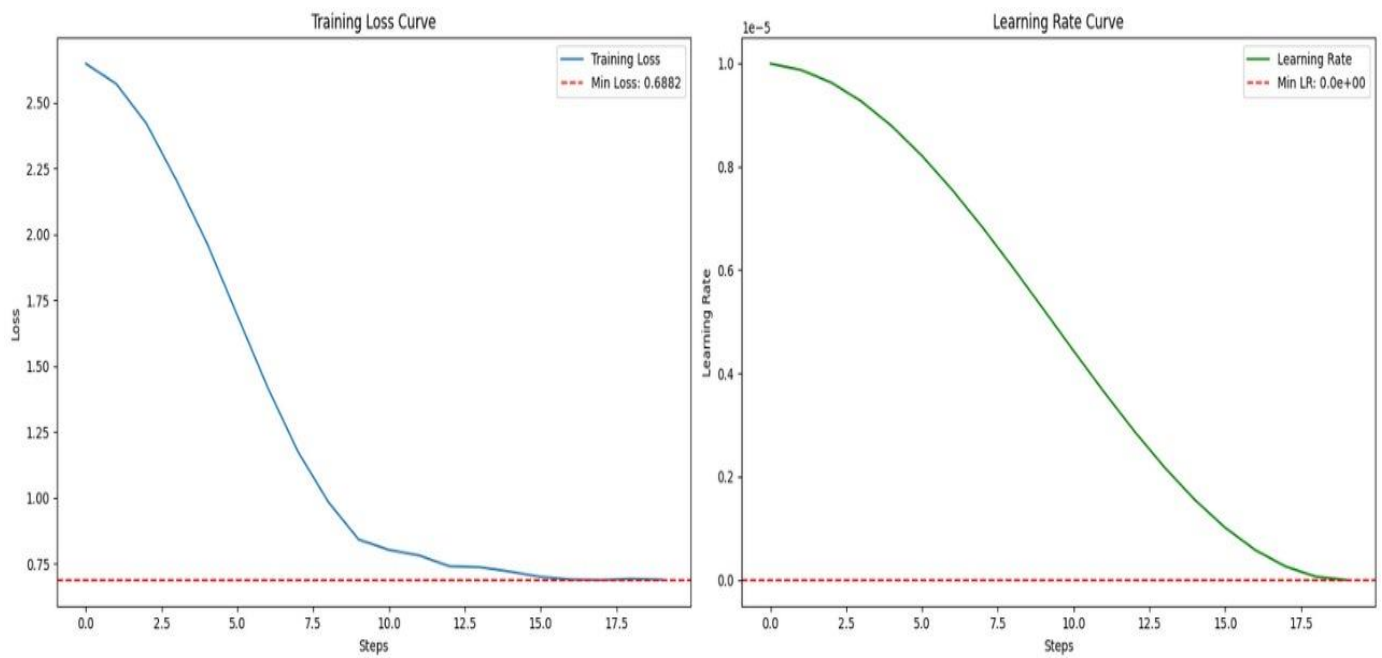


Figure 5- 2 Training Loss VS Learning Rate

- **Data Validation:**
 - We thoroughly validated the psychotherapy dataset used to fine-tune the large language model. This involved data cleaning, checking for inconsistencies, and ensuring its relevance to the therapeutic domain.

- **Training Loss Curve:**
 - **Initial Loss:** The training loss begins at a high value of around 2.5.
 - **Decline Phase:** There is a rapid decline in the training loss in the initial steps, showing that the model quickly learns to fit the training data.
 - **Plateau Phase:** After about 10 steps, the training loss curve begins to plateau, stabilizing at a value just below 0.75. This indicates that the model's learning rate has reached a point where improvements in training loss are minimal.

- **Validation Loss Curve:**
 - **Initial Loss:** Like the training loss, the validation loss also starts at approximately 2.5.
 - **Decline Phase:** The validation loss decreases rapidly in the initial stages, suggesting good generalization to the validation data.
 - **Plateau Phase:** The validation loss curve also plateaus around step 10, stabilizing at a value close to 0.75. This indicates that the model maintains consistent performance on unseen data as training progresses.

- **Learning Rate Curve (Right Plot):**
 - **Initial High Learning Rate:** The learning rate starts at a relatively high value of 1×10^{-5} . This high initial rate facilitates rapid learning and significant updates to the model parameters.
 - **Decay Phase:** The learning rate decreases progressively over the training steps. This decay helps in reducing the size of updates as the training progresses, allowing for more precise adjustments.
 - **Final Phase:** By step 18, the learning rate approaches zero, corresponding to the plateau phase in the training loss curve. This indicates the end of major updates and the fine-tuning of the model.

- **Comparative Analysis:**
 - **Convergence:** Both training and validation loss curves exhibit a similar pattern: a rapid initial decline followed by a plateau. This pattern is typical of well-trained models and indicates that the model is learning effectively and converging.
 - **Stability:** The final loss values for both training and validation are low and similar, suggesting that the model is not overfitting (performing well on training data but poorly on validation data) or underfitting (performing poorly on both training and validation data).
 - **Generalization:** The alignment of the training and validation loss curves suggests that the model's improvements on the training data are translating well to the validation data, indicating good generalization.

- **Analysis of Training Loss vs. Learning Rate(in figure)**
 - **High Learning Rate Phase:** The initial high learning rate facilitates a steep decline in the training loss, enabling the model to quickly improve its performance.
 - **Decay Phase:** As the learning rate decreases, the rate of loss reduction slows down, allowing for finer adjustments and preventing overshooting.
 - **Plateau Phase:** The learning rate approaching zero aligns with the plateau in the training loss curve, indicating stabilization and convergence of the model.
- **Query Response Evaluation:**
 - The model's responses to various user queries were evaluated for accuracy, coherence, and alignment with therapeutic principles. This ensured the chatbot's ability to provide informative and helpful responses within the context of a therapeutic conversation.
- **Edge Case Testing:**
 - We tested the model's behavior with unusual or nonsensical user queries to assess its robustness and ability to handle unexpected inputs. This ensured the chatbot's functionality remained stable even in unforeseen circumstances.

5.3 *Firebase Testing:*

- **Authentication Testing:**
 - We rigorously tested the user authentication functionalities within Firebase to ensure secure login, data privacy, and proper user management. This included testing for potential vulnerabilities like unauthorized access or data breaches.
- **Database Testing:**
 - The Firebase database underwent testing for data integrity, consistency, and efficient data retrieval. This ensured the chatbot's access to relevant information and seamless data storage within the application.
- **Performance Testing:**
 - We tested Firebase's performance under simulated load to assess its ability to handle user requests efficiently. This ensured the application's scalability and responsiveness under increased user traffic.

5.4 *Flask Server Testing:*

- **API Testing:**
 - We employed automated testing tools to test the functionality and performance of the Flask server APIs. This ensured the APIs accurately processed requests from the mobile application and delivered appropriate responses.
- **Security Testing:**
 - The server underwent security testing to identify and address potential vulnerabilities. This included testing for common security threats like injection attacks and unauthorized access attempts.
- **Load Testing:**
 - Like Firebase, we conducted load testing on the Flask server to assess its ability to handle concurrent user requests and maintain service stability. This ensured the application's responsiveness even under high user traffic.

5.5 *Integration Testing:*

Following individual component testing, we performed comprehensive integration testing to ensure all components functioned seamlessly when working together. This involved testing user interactions within the mobile application, data flow between the application and backend services, and the overall user experience.

By following these comprehensive testing procedures, we were confident in the functionality, stability, and user-friendliness of the developed mobile application with its AI chatbot psychotherapist.

5.6 *Benchmarking with Large Language Models:*

- We compared the chatbot's responses to user queries with those generated by established large language models like Chat-GPT and Gemini.
- This benchmarking assessed the model's ability to generate human-like text alongside its capacity to deliver therapeutic value within its responses.
- The evaluation focused on aspects like coherence, grammatical accuracy, and the model's ability to understand and respond to the emotional context of user queries.

5.7 Human Evaluation:

- We conducted a user study where participants interacted with the chatbot and provided feedback on the quality of its responses. This feedback helped us gauge the model's effectiveness in fostering a sense of trust, empathy, and user comfort during the therapeutic conversation.
- Additionally, we presented the chatbot's responses to specialists in the field of psychotherapy, maintaining anonymity throughout the process. Their agreement on the quality of the responses provided valuable insights into the model's alignment with therapeutic best practices and its ability to offer meaningful support.

5.8 Combined Analysis:

The results were used to refine the model's training data and fine-tuning process. By iteratively comparing the chatbot's responses with established models and human experts, we ensured the model's ability to provide informative, supportive, and therapeutically relevant responses to user queries. This comprehensive evaluation process provided strong evidence for the chatbot's potential as a valuable tool for mental health support.

Chapter 6: Business Model

Chapter 6: BUSINESS MODEL

6.1 *Key partners:*

- **Healthcare Providers:**
 - Collaborate with mental health professionals and therapists to integrate AI therapy into their practices.
- **Technology Partners:**
 - Partner with technology companies to enhance AI capabilities or integrate with existing mental health platforms.
- **Research Institutions:**
 - Collaborate with academic and research institutions to advance the field of AI therapy and validate the system's effectiveness.

6.2 *Key activities:*

- **Research and Development:**
 - Continuously improve and update the AI system's algorithms and machine learning models.
- **Data Analysis:**
 - Analyze vast amounts of data to generate personalized therapy recommendations and problem-solving strategies.
- **User Engagement:**
 - Develop conversational interfaces and natural language processing capabilities for interactive conversations with users.

6.3 *Key resources:*

- **AI Technology:**
 - Utilize advanced algorithms, machine learning models, and natural language processing capabilities.
- **Data Infrastructure:**
 - Establish a secure and scalable infrastructure to handle large volumes of user data.
- **Expertise:**
 - Employ a team of AI specialists, data scientists, therapists, and mental health professionals.

6.4 *Value propositions:*

- **Personalized Therapy:**
 - Provide tailored therapy recommendations and problem-solving capabilities based on advanced algorithms and machine learning techniques.
- **Accessibility:**
 - Ensure easy access to mental health support through an AI system, available anytime and anywhere.
- **Empathetic Conversations:**
 - Engage in interactive conversations with users, actively listening to their concerns and providing empathetic responses.
- **Privacy and Confidentiality:**
 - Maintain a secure and confidential environment for users, adhering to strict data privacy and ethical guidelines.

6.5 *Customer relationships:*

- **Personalization:**
 - Provide personalized therapy recommendations and problem-solving strategies tailored to each user's needs.
- **User Support:**
 - Offer customer support channels for inquiries, technical assistance, and user feedback.
- **Continuous Improvement:**
 - Regularly update and enhance the AI therapy system based on user feedback and emerging research.

6.6 *Channels:*

- **Direct Sales:**
 - Market and sell the AI therapy system directly to individuals through online platforms and mobile applications.
- **Partnerships:**
 - Establish partnerships with healthcare providers and organizations to offer AI therapy as part of their mental health services.
- **Online Platforms:**
 - Integrate the AI therapy system with existing mental health platforms or teletherapy services.

6.7 *Customer segments:*

- Individuals seeking mental health support.
- Healthcare providers and organizations offering mental health services.
- Employee assistance programs (EAPs) and human resources departments.

6.8 *Cost structure:*

- **Research and Development:**
 - Allocate resources for ongoing research, development, and improvement of the AI therapy system.
- **Infrastructure:**
 - Invest in secure data storage, computational resources, and IT infrastructure.
- **Talent:**
 - Attract and retain skilled professionals, including AI specialists, data scientists, therapists, and mental health professionals.
- **Marketing and Sales:**
 - Allocate budget for marketing and sales activities to promote the AI therapy system and acquire customers.
- **Model processing:**
 - Cost for fine tuning the large language model and testing it using it's API tokens.

6.9 Revenue stream:

- **Subscription Model:**

- Charge users a recurring subscription fee for access to the AI therapy system.

- **Licensing Model:**

- Offer licensing agreements to healthcare providers or organizations for integrating the AI therapy system into their services.

- **Advertising Model:**

- Show advertisements from other companies when customers use the application.

6.10 SWOT Analysis:

6.10.1 Strengths:

- Advanced algorithms and machine learning techniques for personalized therapy recommendations.
- Problem-solving capabilities to address various mental health issues.
- Robust security measures and adherence to ethical guidelines.

6.10.2 Weaknesses:

- Reliance on accurate data input from users for effective recommendations.
- Potential challenges in user acceptance and trust in AI-based therapy.

6.10.3 Opportunities:

- Growing demand for accessible and personalized mental health support.
- Expansion into new market segments or geographical regions.
- Integration with existing mental health platforms or partnerships with healthcare providers.

6.10.4 Threats:

- Data privacy and security concerns.
- Regulatory compliance requirements.
- Competition from other AI therapy providers.

Chapter 7: Conclusion And Future Work

Chapter 7: CONCLUSION AND FUTURE WORK:

7.1 Conclusion:

- This graduation project has explored the potential of AI in the field of mental healthcare. We developed a mobile application featuring an AI chatbot psychotherapist.
- This chatbot, powered by a large language model fine-tuned on psychotherapy data, allows users to engage in therapeutic conversations, expanding the accessibility of mental health support.

7.2 Future Work:

- **Multilingual Support:**
 - Expanding the application to support Arabic, alongside English or other languages, would significantly broaden its reach and cater to a wider audience. This would involve retraining the language model on Arabic psychotherapy data.
- **Enhanced Functionality:**
 - Incorporating additional features like mood tracking, journaling prompts, or psychoeducational resources within the app. These features can enrich the user experience and provide a more comprehensive approach to mental well-being.
- **Advanced AI Integration:**
 - Integration of more sophisticated AI techniques like sentiment analysis to personalize the chatbot's responses and adapt to the user's emotional state.
- **Scalability and Infrastructure:**
 - Upgrading the server and database infrastructure will be crucial as the user base grows. This ensures smooth performance and data security.
- **Clinical Validation:**
 - Collaborating with mental health professionals to conduct clinical trials to assess the effectiveness of the chatbot in real-world therapeutic setting

REFERENCES

- (Akshit Mehra, 2023) Labellerr, Hands-on with fine-tuning LLM. Retrieved from <https://www.labellerr.com/blog/hands-on-with-fine-tuning-llm/>
- (Arun Brahma, 2023) Hugging Face, Mental Health Chatbot Dataset. Retrieved from https://huggingface.co/datasets/heliosbrahma/mental_health_chatbot_dataset
- (Dart official website, 2023) Dart, Guides. Retrieved from <https://dart.dev/guides>
- (Firebase documentation, 2023) Google, Firebase Authentication. Retrieved from <https://firebase.google.com/docs/auth>
- (Flask official website, 2023) Pallets Projects, Flask QuickStart. Retrieved from <https://flask.palletsprojects.com/en/3.0.x/quickstart/>
- (Maria Habib et al, 2021) ResearchGate, A Predictive Text System for Medical Recommendations in Telemedicine: A Deep Learning Approach in the Arabic Context. Retrieved from https://www.researchgate.net/publication/352269276_A_Predictive_Text_System_for_Medical_Recommendations_in_Telemedicine_A_Deep_Learning_Approach_in_the_Arabic_Context
- (Miguel Cazorla, 2023) MDPI, Journal of Imaging. Retrieved from <https://www.mdpi.com/2673-2688/4/3/35>
- (Mohammed Abdelhay et al, 2023) Springer, Article on Social Network Analysis and Mining. Retrieved from <https://link.springer.com/article/10.1007/s13278-023-01077-w>
- (Technology Innovation Institute, 2023) FalconLLM, Falcon Language Model. Retrieved from <https://falconllm.tii.ae/>
- (Ying Shen et al, 2018) ResearchGate, IDDAT: An Ontology-Driven Decision Support System for Infectious Disease Diagnosis and Therapy. Retrieved from https://www.researchgate.net/publication/329414473_IDDAT_An_Ontology-Driven_Decision_Support_System_for_Infectious_Disease_Diagnosis_and_Therapy

Appendices

APPENDIX A: CODE OF THE BACKEND SERVER

```
from flask import Flask, request, jsonify, session
from flask_cors import CORS
from flask_session import Session
import os
from FB_AUTH import log_in, sign_up
from ML_MODEL import Model

import warnings

app = Flask(__name__)
CORS(app) # This will enable CORS for all routes

# Configure server-side sessions
app.config['SESSION_TYPE'] = 'filesystem'
app.config['SECRET_KEY'] = 'your_secret_key'
Session(app)

# model = None

@app.route('/chat', methods=['POST'])
def chat():
    if 'history' not in session:
        session['history'] = []

    text = request.data.decode('utf-8')
    print(f"Received text: {text}")

    session['history'].append(text)
    response = model.Chain(text)
    # return jsonify({'response': response, 'history': session['history']})
    return response

@app.route('/signin', methods=['POST'])
def signin():
    json_data = request.get_json()
    print(json_data)
    email = json_data.get("email", "")
    password = json_data.get("password", "")
    result = log_in(email, password)
    print(result)
    return result
```

```

@app.route('/signup', methods=['POST'])
def signup():
    json_data = request.get_json()
    email = json_data.get('email')
    password = json_data.get('password')
    fullName = json_data.get('fullName')
    userName = json_data.get('userName')

    result = sign_up(email, password, fullName)
    return result

if __name__ == '__main__':
    warnings.filterwarnings("ignore", category=UserWarning)
    if os.environ.get("WERKZEUG_RUN_MAIN") == 'true':
        os.environ["TF_ENABLE_ONEDNN_OPTS"] = '0'
        model = Model()
        model.load_pretrained()
    app.run(debug=True, port=8090)

```

APPENDIX B: CODE OF FIREBASE

```
import os
from dotenv import load_dotenv
import pyrebase
import firebase_admin
from firebase_admin import credentials
from firebase_admin import db

# Load environment variables from .env file
load_dotenv()

# Access the environment variables
FIREBASE_CONFIG = {
    'apiKey': os.getenv('FIREBASE_API_KEY'),
    'authDomain': os.getenv('FIREBASE_AUTH_DOMAIN'),
    'databaseURL': os.getenv('FIREBASE_DATABASE_URL'),
    'projectId': os.getenv('FIREBASE_PROJECT_ID'),
    'storageBucket': os.getenv('FIREBASE_STORAGE_BUCKET'),
    'messagingSenderId': os.getenv('FIREBASE_MESSAGING_SENDER_ID'),
    'appId': os.getenv('FIREBASE_APP_ID'),
    'measurementId': os.getenv('FIREBASE_MEASUREMENT_ID')
}

#Configure and Connex to Firebase
def log_in(email,password):

    firebase=pyrebase.initialize_app(FIREBASE_CONFIG)
    auth=firebase.auth()

    #Login function

    print("Log in...")
    try:
        auth.sign_in_with_email_and_password(email, password)
        print("Successfully logged in!")
        result = "Successfully logged in!"
    except:
        print("Invalid email or password")
        result = "Invalid email or password"

    return result
```

```

#Signup Function
def sign_up(email,password,fullName):
    firebase=pyrebase.initialize_app(FIREBASE_CONFIG)
    auth=firebase.auth()

    print("Sign up...")
    auth.create_user_with_email_and_password(email, password)
    # login()
    # Initialize Firebase with your service account key file and database URL
    try:
        # Load the Firebase credentials from the JSON file
        cred = credentials.Certificate("key.json")

        # Initialize the Firebase app
        firebase_admin.initialize_app(cred, {
            'databaseURL': os.getenv("FIREBASE_DATABASE_URL")
        })

        # Get a reference to the 'chatbot1' node
        ref = db.reference('/chatbot1')

        # Extract the username from the email
        username = email.split("@")[0]

        # Set the user details in the database
        users_ref = ref.child(username).set({
            'fullName': fullName,
            'chat_history': None
        })

        # Set the 'chat_history' attribute for the user to None
        # users_ref.set({
        #     'chat_history': "ffffff"
        # })
        print("Chat history created successfully.")

    except Exception as e:
        print("Error:", e)

    # print("Email already exists")
    return "Successfully Signing up..."

```

APPENDIX C: CODE OF FINE TUNING LARGE LANGUAGE MODEL

```
import torch
import time
from huggingface_hub import notebook_login
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig,
AutoTokenizer, GenerationConfig
from peft import LoraConfig, get_peft_model, PeftConfig, PeftModel, prepare_model_for_kbit_training
from transformers import TrainingArguments
from trl import SFTTrainer
#ignore warnings
import warnings
from transformers.integrations import TensorBoardCallback
from tqdm import tqdm
warnings.filterwarnings("ignore")
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/COLAB_DATA/moris_data_975_row.csv')
df_test = pd.read_csv('/content/drive/MyDrive/COLAB_DATA/test_data.csv')
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch
# Defining the name of the Falcon model
model_name = "tiiuae/falcon-7b"

# Configuring the BitsAndBytes quantization
bnb_config = BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.float16,
)

# Loading the Falcon model with quantization configuration
model = AutoModelForCausalLM.from_pretrained(
model_name,
quantization_config=bnb_config,
trust_remote_code=True
)

# Disabling cache usage in the model configuration
model.config.use_cache = False
```

```

# Load the tokenizer for the Falcon 7B model with remote code trust
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)

# Set the padding token to be the same as the end-of-sequence token
tokenizer.pad_token = tokenizer.eos_token

from datasets import DatasetDict, Dataset

# Define the formatting function
def format_input(example: dict) -> dict:
    example['conversation'] = f"### You are a psychotherapist for people who will ask you for help to get rid of what is bothering them, and you must give them advice and make sure whether their psychological state is better or not.
    'patient:{example['Question']}' 'therapist:{example['Answer']}' ""
    return example

# Apply the formatting function
dataset = Dataset.from_pandas(pd.DataFrame(df))
datasets = dataset.map(format_input, batched=False)

# Print the result
print(datasets)

dataset_test = Dataset.from_pandas(pd.DataFrame(df_test))

from sklearn.model_selection import train_test_split
train_test_split = datasets.train_test_split(test_size=0.2, shuffle=True, seed=42)

# Import the necessary module for LoRA configuration
from peft import LoraConfig

# Define the parameters for LoRA configuration
lora_alpha = 32
lora_dropout = 0.05
lora_r = 32

```

```

# Create the LoRA configuration object
peft_config = LoraConfig(
    lora_alpha=lora_alpha,
    lora_dropout=lora_dropout,
    r=lora_r,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "query_key_value",
        "dense",
        "dense_h_to_4h",
        "dense_4h_to_h",
    ]
)

from transformers import TrainingArguments
# Define the directory to save training results
output_dir = "./results"

# Set the batch size per device during training
per_device_train_batch_size = 4

# Number of steps to accumulate gradients before updating the model
gradient_accumulation_steps = 4

# Choose the optimizer type (e.g., "paged_adamw_32bit")
optim = "paged_adamw_32bit"

# Interval to save model checkpoints (every 10 steps)
save_steps = 10

# Interval to log training metrics (every 10 steps)
logging_steps = 10

# Learning rate for optimization
learning_rate = 1e-5

# Maximum gradient norm for gradient clipping
max_grad_norm = 0.3

# Maximum number of training steps
max_steps = 200

# Warmup ratio for learning rate scheduling
warmup_ratio = 0.03

# Type of learning rate scheduler (e.g., "constant")
lr_scheduler_type = "cosine"

```

```

# Create a TrainingArguments object to configure the training process
training_arguments = TrainingArguments(
    output_dir=output_dir,
    per_device_train_batch_size=per_device_train_batch_size,
    gradient_accumulation_steps=gradient_accumulation_steps,
    optim=optim,
    save_steps=save_steps,
    logging_steps=logging_steps,
    learning_rate=learning_rate,
    fp16=True, # Use mixed precision training (16-bit)
    max_grad_norm=max_grad_norm,
    max_steps=max_steps,
    warmup_ratio=warmup_ratio,
    group_by_length=True,
    lr_scheduler_type=lr_scheduler_type,
    load_best_model_at_end=True,
    save_strategy="steps", # Save model at intervals
    evaluation_strategy="steps", # Evaluate at intervals
)

# Import the SFTTrainer from the TRL library
from trl import SFTTrainer

# Set the maximum sequence length
max_seq_length = 150

# Create a trainer instance using SFTTrainer
trainer = SFTTrainer(
    model=model,
    train_dataset=train_test_split["train"],
    eval_dataset=train_test_split["test"],
    peft_config=peft_config,
    dataset_text_field="conversation",
    max_seq_length=max_seq_length,
    tokenizer=tokenizer,
    args=training_arguments,
    callbacks=[TensorBoardCallback()]
)

for name, module in trainer.model.named_modules():
    if "norm" in name:
        module = module.to(torch.float32)

trainer.train()

```

APPENDIX D: CODE OF LOAD LARGE LANGUAGE MODEL

```
import torch
from peft import PeftConfig, PeftModel
import re
from typing import List
from langchain import PromptTemplate
from langchain.chains import ConversationChain
from langchain.chains.conversation.memory import ConversationBufferWindowMemory
from langchain_community.llms import HuggingFacePipeline
from langchain.schema import BaseOutputParser
from langchain.memory import ConversationSummaryMemory, ChatMessageHistory
from langchain.chains.conversation.memory import ConversationBufferMemory
import warnings
import os
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    StoppingCriteria,
    StoppingCriteriaList,
    # GenerationConfig,
    pipeline)
warnings.filterwarnings("ignore", category=UserWarning)
# Set environment variable to turn off oneDNN custom operations
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
class Model():
    def __init__(self):
        self.falcon_model = None
        self.falcon_model_peft = None
        self.falcon_tokenizer = None
        self.gen_config = None
        self.chain = None
    def load_pretrained(self):
        # Creates an instance of BitsAndBytesConfig with specific settings for quantization.
        # BitsAndBytesConfig for quantization settings
        bnb_config = BitsAndBytesConfig(
            # load_in_8bit=True,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_use_double_quant=True,
            bnb_4bit_compute_dtype=torch.float16,
            llm_int8_enable_fp32_cpu_offload=True )
```

```

# Load and configure the Falcon model and tokenizer
config = PeftConfig.from_pretrained("MODEL_FILES")
self.falcon_model = AutoModelForCausalLM.from_pretrained(
    config.base_model_name_or_path,
    return_dict=True,
    quantization_config=bnb_config,
    low_cpu_mem_usage = True,
    trust_remote_code=True,
    device_map="cuda:0"
)
#-----
self.falcon_tokenizer = AutoTokenizer.from_pretrained(
    config.base_model_name_or_path,
    trust_remote_code=True
)
self.falcon_tokenizer.pad_token = self.falcon_tokenizer.eos_token
#-----
self.falcon_model_peft = PeftModel.from_pretrained(self.falcon_model,"MODEL_FILES")

# # set the generation configuration params
self.gen_config = self.falcon_model_peft.generation_config
self.gen_config.max_new_tokens = 1024
self.gen_config.temperature = 0
self.gen_config.top_p = 0.5
self.gen_config.num_return_sequences = 1
self.gen_config.repetition_penalty= 1.2
self.gen_config.pad_token_id = self.falcon_tokenizer.pad_token_id
self.gen_config.eos_token_id = self.falcon_tokenizer.eos_token_id

self.Create_Chain()

def Create_Chain(self):

    generation_pipeline = pipeline(
        model=self.falcon_model_peft,
        tokenizer=self.falcon_tokenizer,
        return_full_text=True,
        task="text-generation",
        # stopping_criteria=stopping_criteria,
        generation_config=self.gen_config,
        do_sample = False,
        use_cache = True
    )

    llm = HuggingFacePipeline(pipeline=generation_pipeline)
    template = """
    You're a psychologist, and a patient has reached out to you for help.\n\n{history}\n\npatient:
    {input}\npsychologist:""".strip()

```

```

prompt = PromptTemplate(input_variables=["history","input"], template=template)

# memory = ConversationBufferMemory()
# memory = ConversationSummaryMemory(llm=self.llm)

# We set a low k=2, to only keep the last 2 interactions in memory
memory = ConversationBufferWindowMemory(
    memory_key="history", k=4, return_only_outputs=True
)
self.chain = ConversationChain(
    llm=llm,
    memory = memory,
    prompt=prompt,
    # output_parser=self.LastWordFinder(),
    verbose=True,
)
def get_last_psychologist_response(self, conversation):
    """
    This function takes a conversation string and returns the last response from the psychologist,
    without the "psychologist:" prefix.
    Parameters:
    conversation (str): The full conversation as a string.
    Returns:
    str: The last response from the psychologist without the prefix.
    """
    # Split the conversation into lines
    lines = conversation.split('\n')
    # Initialize variables to track the last psychologist response
    last_psychologist_response = []
    current_response = []
    is_psychologist = False
    # Iterate through the lines to find and capture the psychologist's responses
    for line in lines:
        if line.startswith('psychologist:'):
            if current_response:
                last_psychologist_response = current_response
                current_response = []
            is_psychologist = True
            # Append the line without the "psychologist:" prefix
            current_response.append(line[len('psychologist:'):].strip())
        elif line.startswith('patient:'):
            is_psychologist = False
        elif is_psychologist:
            current_response.append(line)
    if current_response:
        last_psychologist_response = current_response

    return '\n'.join(last_psychologist_response)

```

```

def Chain(self,text):
    conv = self.chain.predict(input=text)
    return self.get_last_psychologist_response(conv)
def get_response_falcon(self,input):
    template = ""
    You're a psychologist, and a patient has reached out to you for help.\n\n\npatient:
{input}\npsychologist:"".strip()
    prompt = template.format(input=input)

    falcon_encoding = self.falcon_tokenizer(prompt, return_tensors="pt").to(self.falcon_model.device)

    with torch.inference_mode():
        falcon_outputs = self.falcon_model.peft.generate(
            input_ids = falcon_encoding.input_ids,
            attention_mask = falcon_encoding.attention_mask,
            generation_config = self.gen_config
        )

        falcon_text_output = self.falcon_tokenizer.decode(falcon_outputs[0].cuda(),
skip_special_tokens=True)
        return falcon_text_output

```